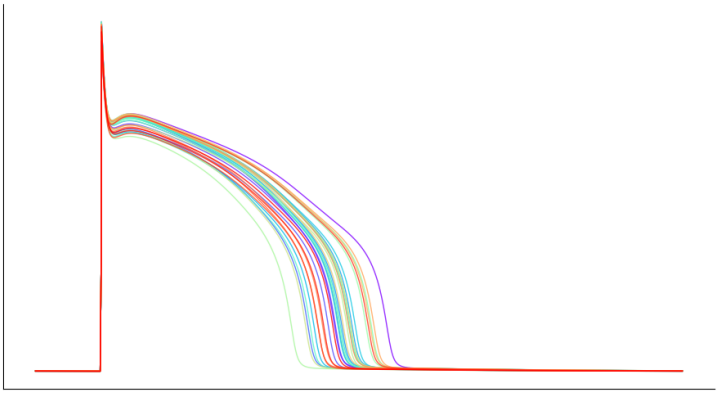
Automatic Translation of Cardiac Ionic Models to GPGPU



Joseph Simkin

Word Count: 10,000 (using texcount)

Honour School of Computer Science (Part C)

Trinity Term 2026

Contents

1 Introduction **1**

1.1 Background 1

1.2 Existing Tools and Limitations 2

1.3 Contributions 3

2 Numerical Methods for Cardiac Cell Models **4**

2.1 The Luo–Rudy 1991 Cell Model 4

2.2 The Forward Euler Method 5

2.3 The Backward Euler Method 5

2.4 The CVODE Solver 8

2.5 Block-Structured Batched Systems 9

3 Software Architecture and Build System **11**

3.1 The Existing Chaste Single Cell Workflow 11

3.2 GPU Integration into Chaste 17

3.3 Integration of GPU-accelerated CVODE 20

3.4 Chaste Build System 21

4 Generated Device Code for Cardiac Cell Models **25**

4.1 GPU-Compatible Generated Cell Models 25

4.2 Code Generation for the Forward Euler Method 26

4.3 Code Generation for the Backward Euler Method 28
4.4 Code Generation for the CVODE Solver 30
4.5 Device-Side Support Utilities 33
4.5.1 ChasteDevice 33
4.5.2 DeviceStimulusFunctor 34

5 Automatic Code Generation for Cardiac Cell Models 39
5.1 Integrating Device Code into Generated Cell Classes 39
5.2 Device Code Generation for Model-Specific Functions 41
5.3 Device-Specific Expression Generation 43
5.4 CVODE Device Code Generation 47

6 CUDA Execution Model for Cardiac Cell Simulation 49
6.1 Thread Execution Model 49
6.2 Memory Model 51
6.3 Hardware constraints 53
6.4 CUDA Streams 53

7 GPU Time-Stepping Solvers 54
7.1 Integration of Generated Device Code into the GPU Solver 54
7.2 Host-Side Integration and Data Management 60
7.3 Backward Euler solver 63
7.4 GPU-Enabled CVODE Integration 64

8 Results and Discussion 68
8.1 GPU Infrastructure Validation 68
8.2 Numerical Stability and Timestep Analysis 69
8.3 GPU Performance Evaluation 73

8.3.1 CVODE validation and performance 76

9 Conclusions **78**

9.1 Contributions and Key Findings 78

9.2 Limitations and Reflections 79

9.3 Future Work 80

A Example Generated GPU Cell Model Code **81**

B Implementation of the GPU Newton Solver **86**

C Implementation of the GPU-Enabled CVODE Solver **89**

Bibliography **93**

Chapter 1

Introduction

1.1 Background

Cardiac electrophysiology modelling has been studied extensively since the seminal work of Alan Hodgkin and Andrew Huxley in the 1950s [12], and has since developed into a mature and widely used field. Several high-performance simulation frameworks have been developed and are now routinely used in both clinical and computational research [22, 6, 2]. At their core, these frameworks simulate systems of differential equations that describe the dynamics of a single cardiac cell, with whole-tissue models constructed by coupling many such cells together. These cellular models typically take the form

$$\frac{d\mathbf{y}}{dt} = \mathbf{f}(\mathbf{y}, V), \qquad C_m \frac{dV}{dt} = -I_{\text{ion}}(\mathbf{y}, V). \tag{1.1}$$

where $\mathbf{y}$ denotes the vector of ionic and gating state variables, V is the transmembrane potential, C_m is the membrane capacitance, and $I_{\text{ion}}(\mathbf{y}, V)$ is the total ionic current across the cell as a function of these quantities [16].

These models are used for two main purposes. First, in single-cell studies for drug development, they are used to investigate the effect of pharmaceutical compounds by modifying the ionic currents that contribute to $I_{\text{ion}}(\mathbf{y}, V)$. Drug action is typically represented by reducing the conductance of specific current components (e.g. $I_{\text{Na}}, I_{\text{Kr}}, I_{\text{Ca}}$), thereby altering their contribution to the total ionic current and, consequently, the evolution of the membrane potential. In practice, large numbers of independent simulations are performed across different parameter sets to capture physiological variability and assess drug effects under a range of conditions.

Second, in tissue and whole-heart simulations, these same single cell models are in-

stantiated across a computational mesh and coupled to a diffusion equation. This results in repeated local ODE solves (one per mesh element) interleaved with PDE updates. In both settings, the dominant computation is therefore large batches of independent ODE solves with identical structure but differing states and parameters, making these workloads embarrassingly parallel and well suited to GPU architectures. Despite this, efficiently exploiting GPUs for general cardiac cell models remains a significant challenge.

The importance of simulation based on these models has only increased with the regulatory acceptance of in silico computational modelling into the drug development pipeline [25]. Improved understanding of cardiac electrophysiology has led to increasingly complex models, advancing from the original 4-variable Hodgkin–Huxley model [12] to the state-of-the-art 44-variable ToR-ORd model [24], with many intermediate models developed along the way. The increasing number and complexity of these models has motivated the development of CellML, an XML-based format for model specification, which has been widely adopted by the community [15]. CellML has been instrumental in enabling the widespread use of these models.

1.2 Existing Tools and Limitations

Several tools have been developed to automate the translation of CellML files into executable code. These tools can be broadly grouped into two categories: generators and simulation engines. Generators translate CellML files into code for model-specific functions, such as `EvaluateYDerivates`, used to solve the system with a particular numerical method. Simulation engines use these functions together with initial conditions to solve the system over time.

Existing frameworks typically provide tools from one or both of these categories. OpenCOR [10], Myokit [5], and Chaste [6] provide both a generator and a simulation engine for CPU execution, with varying degrees of parallelism. Myokit additionally supports GPU code generation via CUDA and OpenCL, although its simulation engine is limited to cable simulations for OpenCL. MonoAlg3d [2] provides a high-performance simulation engine with CPU and GPU support, but only for NVIDIA GPUs and without a corresponding generator. Consequently, extending MonoAlg3d to new models requires manual translation of differential equations into C/CUDA, a process that is both time-consuming and error-prone. Other open-source frameworks

exist [22], but none provide a general solution combining automated code generation with GPU-capable simulation for arbitrary CellML models.

This gap motivates the development of a system capable of automatically generating and efficiently executing general CellML models on GPU architectures.

1.3 Contributions

This project extends the Chaste cardiac electrophysiology framework [6], a large, modular, open-source platform for computational physiology supporting cardiac, tissue, and multi-physics simulations. Although Chaste is no longer among the most widely adopted cardiac electrophysiology frameworks [22], it remains a mature and well-tested codebase with extensive support for CPU-based cardiac cell modelling.

Within Chaste, CellML models are translated into C++ using `chaste-codegen` [11], a Python-based tool which produces cell-model-specific functions for evaluating systems of differential equations within numerical solvers. This work extends both the code generation pipeline and the associated solver infrastructure to support efficient execution of cardiac cell models on CUDA devices [18]. Our primary contributions are:

- A user-friendly CUDA-enabled workflow within Chaste for cardiac cell simulations (Chapter 3).
- A translation of CPU model evaluation code into CUDA device code for cardiac cell models (Chapter 4).
- Modifications to `chaste-codegen` to automatically generate this device code from CellML models (Chapter 5).
- CUDA implementations of Forward and Backward Euler solvers within Chaste, operating on the generated device code (Chapter 7).

In addition, we modernise Chaste's support for the CVODE solver from the SUNDIALS library [9] by extending it to use the CUDA-enabled SUNDIALS infrastructure described in [3] (Section 7.4).

Chapter 2

Numerical Methods for Cardiac Cell Models

2.1 The Luo–Rudy 1991 Cell Model

The Luo–Rudy 1991 model [16] is a representative cardiac cell model of the form introduced in Equation 1.1. Whilst the precise physiological details are not important for understanding the GPU extension developed in this work, many later cardiac cell models share a similar mathematical structure, making the Luo–Rudy system a useful illustrative example.

The model describes the evolution of the transmembrane potential V through

$$\frac{dV}{dt} = -\frac{1}{C}\left(I_i + I_{\text{stim}}\right), \tag{2.1}$$

where C is the membrane capacitance, I_i is the sum of the ionic currents within the cell, and I_{stim} is an externally applied stimulus current used to excite the cell and initiate an action potential, an example of which is shown later in Figure 3.1.

Six of the state variables are gating variables satisfying equations of the form

$$\frac{du_i}{dt} = a_i(V) + b_i(V)u_i, \tag{2.2}$$

where $a_i(V)$ and $b_i(V)$ are voltage-dependent rate functions.

The remaining nonlinear subsystem contains the transmembrane potential together with the intracellular calcium concentration, whose dynamics are given by

$$\frac{d\text{Cai}}{dt} = -10^{-4} i_{\text{si}} + 0.07(10^{-4} - \text{Cai}), \tag{2.3}$$

where i_{si} is the slow inward current.

Before describing the GPU implementations developed in this work, it is useful to introduce the numerical methods and model-specific operations already present in Chaste for integrating models such as the Luo–Rudy 1991 model. The GPU extensions developed in this work preserve these core solver structures, replacing the underlying execution backend rather than redefining the numerical schemes themselves.

2.2 The Forward Euler Method

To solve a system such as the Luo–Rudy 1991 model using the Forward Euler method, we consider the membrane voltage V and remaining state variables together as a single combined state vector $\mathbf{y}$, giving

$$\frac{d\mathbf{y}}{dt} = \mathbf{f}(\mathbf{y}). \tag{2.4}$$

The Forward Euler update is then given by

$$\mathbf{y}_{n+1} = \mathbf{y}_n + \Delta t\, \mathbf{f}(\mathbf{y}_n), \tag{2.5}$$

which advances the solution explicitly using information from the current timestep, requiring only the evaluation of the derivative function $\mathbf{f}(\mathbf{y})$. In Chaste, this is performed by the `EvaluateYDerivatives` routine within model-specific classes such as `CellLuoRudy1991FromCellML`.

Although simple to implement, Forward Euler struggles with stability. Cardiac models exhibit stiff behaviour, particularly during the rapid action potential upstroke, requiring prohibitively small timesteps for stable explicit integration. This stiffness motivates Chaste's inclusion of more stable implicit methods, such as Backward Euler.

2.3 The Backward Euler Method

Given a generic ODE system of the form in Equation 2.4, the Backward Euler method computes $\mathbf{y}_{n+1}$ satisfying

$$\mathbf{y}_{n+1} = \mathbf{y}_n + \Delta t\, \mathbf{f}(\mathbf{y}_{n+1}, t_{n+1}). \tag{2.6}$$

This implicit update is solved by defining the residual

$$R(\mathbf{y}_{n+1}) = \mathbf{y}_{n+1} - \mathbf{y}_n - \Delta t\, \mathbf{f}(\mathbf{y}_{n+1}, t_{n+1}), \tag{2.7}$$

and finding $\mathbf{y}_{n+1}$ such that

$$R(\mathbf{y}_{n+1}) = 0.$$

This nonlinear system is often solved using Newton iteration. Given an approximation $\mathbf{y}^{(k)}$, we linearise $\mathbf{f}$ about this point:

$$\mathbf{f}(\mathbf{y}_{n+1}, t_{n+1}) \approx \mathbf{f}(\mathbf{y}^{(k)}, t_{n+1}) + \frac{\partial \mathbf{f}}{\partial \mathbf{y}}(\mathbf{y}^{(k)}, t_{n+1})(\mathbf{y}_{n+1} - \mathbf{y}^{(k)}) \tag{2.8}$$

Letting $\mathbf{y}_{n+1} = \mathbf{y}^{(k)} + \delta\mathbf{y}^{(k)}$, the residual becomes

$$R(\mathbf{y}^{(k)} + \delta\mathbf{y}^{(k)}) = \mathbf{y}^{(k)} + \delta\mathbf{y}^{(k)} - \mathbf{y}_n - \Delta t \left(\mathbf{f}(\mathbf{y}^{(k)}, t_{n+1}) + \frac{\partial \mathbf{f}}{\partial \mathbf{y}}(\mathbf{y}^{(k)}, t_{n+1})\delta\mathbf{y}^{(k)} \right) \tag{2.9}$$

$$= \mathbf{y}^{(k)} - \mathbf{y}_n - \Delta t\, \mathbf{f}(\mathbf{y}^{(k)}, t_{n+1}) + \left(I - \Delta t \frac{\partial \mathbf{f}}{\partial \mathbf{y}}(\mathbf{y}^{(k)}, t_{n+1}) \right) \delta\mathbf{y}^{(k)} \tag{2.10}$$

Using Equation 2.7, and defining the Jacobian $J^{(k)}$, we write

$$R(\mathbf{y}^{(k)}) = \mathbf{y}^{(k)} - \mathbf{y}_n - \Delta t\, \mathbf{f}(\mathbf{y}^{(k)}, t_{n+1}), \quad J^{(k)} = I - \Delta t \frac{\partial \mathbf{f}}{\partial \mathbf{y}}(\mathbf{y}^{(k)}, t_{n+1}) \tag{2.11}$$

Newton iteration then updates $\mathbf{y}^{(k)}$ until convergence to $\mathbf{y}_{n+1}$:

$$J^{(k)}\, \delta\mathbf{y}^{(k)} = R(\mathbf{y}^{(k)}), \quad \mathbf{y}^{(k+1)} = \mathbf{y}^{(k)} - \delta\mathbf{y}^{(k)} \tag{2.12}$$

This can be particularly efficient when using the previous solution $\mathbf{y}_n$ as the initial estimate $\mathbf{y}^{(0)}$. While implementing this algorithm directly yields a valid method for computing $\mathbf{y}_{n+1}$, the structure of ionic models can be exploited to reduce the system size solved on each iteration [26].

In the Luo–Rudy 1991 model introduced earlier, six of the state variables are gating variables satisfying equations of the form in Equation 2.2. Crucially, the derivative of each gating variable depends only on the current membrane potential V and the variable itself. Treating the membrane potential as fixed over a timestep makes each gating equation linear in its corresponding state variable.

Applying Backward Euler to Equation 2.2 gives

$$u_{i,n+1} = u_{i,n} + \Delta t \left(a_i(V) + b_i(V) u_{i,n+1} \right), \tag{2.13}$$

which may be rearranged explicitly to obtain

$$u_{i,n+1} = \frac{u_{i,n} + \Delta t\, a_i(V)}{1 - \Delta t\, b_i(V)}. \tag{2.14}$$

The existing Chaste implementation therefore uses a semi-implicit strategy. The transmembrane potential is first advanced explicitly using

$$V_{n+1} = V_n - \frac{\Delta t}{C} \left(I_i(\mathbf{y}_n, V_n) + I_{\text{stim}}(t_n) \right), \tag{2.15}$$

after which the gating variables are updated using the closed-form Backward Euler expression in Equation 2.14, and finally the remaining state variables are solved for using Newton's method.

After reordering the state if necessary, group the explicitly updated transmembrane potential together with the gating variables into a vector $\mathbf{u}$, and write the remaining nonlinear variables as $\mathbf{z}$.

The Newton iterate therefore takes the form

$$\mathbf{y}^{(k)} = \begin{pmatrix} \mathbf{u}^*_{n+1} \\ \mathbf{z}^{(k)} \end{pmatrix}, \tag{2.16}$$

where $\mathbf{u}^*_{n+1}$ denotes the precomputed voltage and gating updates.

Substituting this iterate into the residual definition in Equation 2.7 gives

$$R(\mathbf{y}^{(k)}) = \begin{pmatrix} \mathbf{0} \\ R_z(\mathbf{u}^*_{n+1}, \mathbf{z}^{(k)}) \end{pmatrix}, \tag{2.17}$$

since the gating-variable equations have already been satisfied exactly by the closed-form updates.

Writing the Newton system from Equation 2.12 in block form gives

$$\begin{pmatrix} J^{(k)}_{uu} & J^{(k)}_{uz} \\ J^{(k)}_{zu} & J^{(k)}_{zz} \end{pmatrix} \begin{pmatrix} \delta\mathbf{u}^{(k)} \\ \delta\mathbf{z}^{(k)} \end{pmatrix} = \begin{pmatrix} \mathbf{0} \\ R_z(\mathbf{u}^*_{n+1}, \mathbf{z}^{(k)}) \end{pmatrix}. \tag{2.18}$$

Since $\mathbf{u}^*_{n+1}$ is already the exact solution of the gating subsystem, we know that

$$\delta\mathbf{u}^{(k)} = \mathbf{0}.$$

The Newton iteration therefore reduces to

$$J^{(k)}_{zz}\, \delta\mathbf{z}^{(k)} = R_z(\mathbf{u}^*_{n+1}, \mathbf{z}^{(k)}), \quad \mathbf{z}^{(k+1)} = \mathbf{z}^{(k)} - \delta\mathbf{z}^{(k)}. \tag{2.19}$$

Although this sacrifices the unconditional stability properties of a fully implicit Backward Euler solve, treating the gating equations implicitly captures a substantial portion of the stiffness present in many cell models in the form of Equation 1.1, whilst allowing us to solve a much reduced linear system. This observation also motivates the Rush–Larsen method, whose improved stability derives from implicit treatment of the gating variables [23]. Rush–Larsen is also used within MonoAlg3D for GPU execution of modern cardiac cell models such as the ToR–ORd model [2, 24]. While Rush–Larsen is not implemented here, our GPU infrastructure could naturally be extended to support it.

This numerical structure is reflected in existing Chaste Backward Euler classes, such as `CellLuoRudy1991FromCellMLBackwardEuler`. These classes provide an `UpdateTransmembranePotential` method implementing the explicit voltage update in Equation 2.15 for that specific cell model, together with `ComputeOneStepExceptVoltage`, which applies the closed-form gating updates from Equation 2.14 before initiating the reduced Newton iteration in Equation 2.19. The methods `ComputeResidual` and `ComputeJacobian` provide the residual and Jacobian evaluations required during Newton iteration. As with Forward Euler, these model-specific operations are combined with a solver implementation to advance the system in time.

2.4 The CVODE Solver

CVODE, from the SUNDIALS library [9], solves initial value problems of the form in Equation 2.4. Within Chaste, CVODE provides a drop in, more efficient, alternative to the built-in ODE solvers described previously, and is configured to use the Backward Differentiation Formula (BDF) family of methods.

A BDF method approximates the local solution by fitting an interpolation polynomial through previous solution values and enforcing the differential equation at the newest time point. For a method of order k, this gives a multistep equation of the form

$$\mathbf{y}_{n+1} = \frac{1}{\alpha_0}\left(h_n \mathbf{f}(t_{n+1}, \mathbf{y}_{n+1}) - \sum_{i=1}^{k} \alpha_i \mathbf{y}_{n+1-i}\right). \tag{2.20}$$

where h_n is the current timestep and the coefficients α_i depend on the order and recent timestep history. For example, the first-order BDF method is simply Backward Euler. Higher-order BDF methods use more previous solution values to construct a higher-order local approximation.

At each internal timestep, CVODE must solve the nonlinear system implied by Equation 2.20. Writing this system as a residual gives

$$R(\mathbf{y}_{n+1}) = \alpha_0 \mathbf{y}_{n+1} - h_n \mathbf{f}(t_{n+1}, \mathbf{y}_{n+1}) + \sum_{i=1}^{k} \alpha_i \mathbf{y}_{n+1-i} = \mathbf{0}. \tag{2.21}$$

CVODE supports several nonlinear and linear solver configurations for solving systems of this form [9]. In Chaste, CVODE is configured to use Newton iteration together with a dense linear solver. The model-specific derivative function is supplied through `EvaluateYDerivatives`, while an analytic Jacobian may optionally be provided through `EvaluateAnalyticJacobian`.

Unlike the Forward Euler and Backward Euler solvers described above, CVODE controls its own internal timesteps and method order. The user specifies output times and error tolerances, while CVODE estimates the local truncation error at each timestep, rejecting or accepting steps and dynamically adjusting both timestep and method order accordingly.

2.5 Block-Structured Batched Systems

Whilst the GPU implementations of the Forward and Backward Euler methods developed later remain conceptually similar to their CPU counterparts, the GPU interface exposed by a recent version of CVODE [3] leads to a fundamentally different execution model. For the Euler methods, parallelism may be introduced naturally by solving many independent cell models simultaneously, with each GPU thread performing essentially the same operations as a corresponding CPU solve. CVODE, however, internally controls timestep selection, error estimation, and Newton iteration from the host, whilst delegating residual, Jacobian, and linear solver evaluations to externally supplied routines.

Consequently, efficient GPU execution with CVODE is achieved not by launching many independent solver instances, but by combining many uncoupled cell models into a single larger system. This allows the expensive linear algebra and Jacobian evaluations performed during Newton iteration to operate on problem sizes large

enough to efficiently utilise the GPU. The combined system may be written as

$$\mathbf{Y} = \begin{pmatrix} \mathbf{y}_1 \\ \mathbf{y}_2 \\ \vdots \\ \mathbf{y}_N \end{pmatrix}, \tag{2.22}$$

with corresponding right-hand side

$$\mathbf{F}(\mathbf{Y}) = \begin{pmatrix} \mathbf{f}_1(\mathbf{y}_1) \\ \mathbf{f}_2(\mathbf{y}_2) \\ \vdots \\ \mathbf{f}_N(\mathbf{y}_N) \end{pmatrix}. \tag{2.23}$$

Since the cells are independent, the Jacobian of the combined system has block diagonal structure,

$$J = \frac{\partial \mathbf{F}}{\partial \mathbf{Y}} = \begin{pmatrix} J_1 & 0 & \cdots & 0 \\ 0 & J_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & J_N \end{pmatrix}, \tag{2.24}$$

where each block J_i corresponds to the Jacobian of a single cell model.

This formulation allows CVODE's adaptive BDF timestepping and Newton iteration to be combined effectively with GPU execution. However, the entire block system shares a single timestep controller. Consequently, a particularly stiff or unstable cell model may force smaller timesteps for the entire batch, reducing overall computational efficiency.

Chapter 3

Software Architecture and Build System

3.1 The Existing Chaste Single Cell Workflow

Before introducing the CUDA extension, it is useful to share the standard single-cell workflow already supported by Chaste through a representative Luo–Rudy 1991 simulation adapted from the Chaste single-cell tutorial [16].

In this example, we choose to stimulate the Luo–Rudy 1991 model once and simulate the resulting action potential using the Forward Euler method. It is useful to expand the previously introduced Equation 2.1 to see the membrane voltage equation in full detail:

$$\frac{dV}{dt} = -\frac{I_{\text{stim}} + i_{\text{Na}} + i_{\text{si}} + i_K + i_{K1} + i_{Kp} + i_b}{C}. \tag{3.1}$$

As described in Section 2.2, Chaste's Forward Euler implementation treats the membrane voltage together with the remaining state variables as a combined state vector $\mathbf{y}$, solving

$$\frac{d\mathbf{y}}{dt} = \mathbf{f}(\mathbf{y}) \tag{3.2}$$

using the update

$$\mathbf{y}_{n+1} = \mathbf{y}_n + \Delta t\, \mathbf{f}(\mathbf{y}_n). \tag{3.3}$$

In Chaste, this is implemented by combining a numerical solver with the generated class `CellLuoRudy1991FromCellML`, which provides the model-specific `EvaluateYDerivatives` routine defining $\mathbf{f}(\mathbf{y})$. Figure 3.1 shows the complete workflow and resulting action potential.

```cpp
#include "EulerIvpOdeSolver.hpp"
#include "SimpleStimulus.hpp"
#include "LuoRudy1991.hpp"

// Create a solver (applies the Forward Euler update)
boost::shared_ptr<AbstractIvpOdeSolver> p_solver(new EulerIvpOdeSolver());

// Define a stimulus: (magnitude, duration, start time)
boost::shared_ptr<SimpleStimulus> p_stimulus(
    new SimpleStimulus(-25.5, 2.0, 100.0));

// Construct the cell model (provides EvaluateYDerivatives for Luo-Rudy 1991)
boost::shared_ptr<CellLuoRudy1991FromCellML> p_model(
    new CellLuoRudy1991FromCellML(p_solver, p_stimulus));

// Optionally modify model parameters
p_model->SetParameter(
    "membrane_rapid_delayed_rectifier_potassium_current_conductance",
    0.24);

// Simulate for 1s, taking a sample every 0.1ms
OdeSolution solution =
    p_model->Compute(0.0, 1000.0, 0.1);

solution.WriteToFile(
    "TestForwardEulerCells",
    "LuoRudy1991ForwardEuler",
    "ms",
    1,
    false,
    12);
```

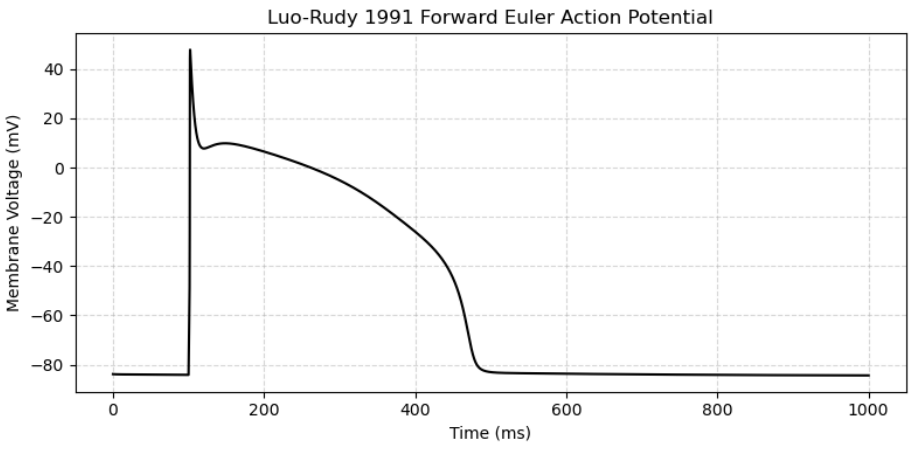


Figure 3.1: Example Chaste Forward Euler workflow using the Luo–Rudy 1991 model, together with the resulting transmembrane potential trace.

The `CellLuoRudy1991FromCellML` class used previously is not implemented manually. Chaste supports many combinations of cell models and numerical methods, each

requiring specialised generated routines as discussed in Chapter 2. Maintaining this cross product of model and solver implementations by hand would be cumbersome and error-prone, so Chaste instead generates them automatically from CellML model specifications using `chaste-codegen`.

Listing 1 shows the CellML description corresponding to Equation 3.1. The important observation is that CellML provides a structured representation of the model equations.

Within Chaste, `chaste-codegen` parses these CellML model descriptions and performs symbolic analysis to identify the state variables, model parameters, and mathematical expressions defining the system. This information is then substituted into the variables of numerical-method-specific Jinja2 [20] templates to generate the corresponding C++ implementations.

Listing 2 shows a simplified version of the template used to generate `EvaluateYDerivatives`, while Listing 3 shows the resulting generated C++ implementation for the Luo–Rudy 1991 model. Together, these listings illustrate the Chaste code-generation pipeline from CellML equations to executable numerical implementations.

This relationship between symbolic model representation and generated code is central to the CUDA extensions introduced later in this work. The generated CUDA device functions follow the same symbolic structure and variable naming conventions as the generated CPU implementations, allowing direct comparison between the two execution backends.

```
<component name="membrane">
    <variable units="millivolt" public_interface="out"
              cmeta:id="membrane_voltage"
              name="V" initial_value="-83.853"/>

    <variable units="microF_per_cm2" name="C" initial_value="1"/>
    <variable units="microA_per_cm2" name="I_stim"
              cmeta:id="membrane_stimulus_current"/>

    <variable units="millisecond" public_interface="in" name="time"/>
    <variable units="microA_per_cm2" public_interface="in" name="i_Na"/>
    <variable units="microA_per_cm2" public_interface="in" name="i_si"/>
    <variable units="microA_per_cm2" public_interface="in" name="i_K"/>
    <variable units="microA_per_cm2" public_interface="in" name="i_K1"/>
    <variable units="microA_per_cm2" public_interface="in" name="i_Kp"/>
    <variable units="microA_per_cm2" public_interface="in" name="i_b"/>

    <math xmlns="http://www.w3.org/1998/Math/MathML">
        <apply>
            <eq/>
            <apply>
                <diff/>
                <bvar><ci>time</ci></bvar>
                <ci>V</ci>
            </apply>
            <apply>
                <times/>
                <apply>
                    <divide/>
                    <apply>
                        <minus/>
                        <cn cellml:units="dimensionless">1</cn>
                    </apply>
                    <ci>C</ci>
                </apply>
                <apply>
                    <plus/>
                    <ci>I_stim</ci>
                    <ci>i_Na</ci>
                    <ci>i_si</ci>
                    <ci>i_K</ci>
                    <ci>i_K1</ci>
                    <ci>i_Kp</ci>
                    <ci>i_b</ci>
                </apply>
            </apply>
        </apply>
    </math>
</component>
```

Listing 1: CellML representation of Equation 3.1.

```
void {{class_name}}::EvaluateYDerivatives(
    double {{free_variable.var_name}},
    const {{vector_decl}} rY,
    {{vector_decl}} rDY)
{
    // Inputs:
    // Time units: millisecond
    {%- for state_var in state_vars %}
    {% if state_var.in_y_deriv %}
    double {{ state_var.var }} = {{ state_var.rY_lookup }};
    // Units: {{ state_var.units }};
    // Initial value: {{ state_var.initial_value }}
    {%- endif %}
    {%- endfor %}

    // Mathematics
    {% for deriv in y_derivative_equations %}
    {%- if deriv.is_voltage %}
    double {{ deriv.lhs }};
    {%- endif %}
    {%- endfor %}

    {%- for deriv in y_derivative_equations %}
    {%- if deriv.in_eqs_excl_voltage %}
    const double {{ deriv.lhs }} = {{ deriv.rhs }}; // {{ deriv.units }}
    {%- endif %}
    {%- endfor %}

    if (mSetVoltageDerivativeToZero)
    {
        {% for deriv in y_derivative_equations %}
        {%- if deriv.is_voltage %}
        {{ deriv.lhs }} = 0.0;
        {%- endif %}
        {%- endfor %}
    }
    else
    {
        {% for deriv in y_derivative_equations %}
        {% if not deriv.in_eqs_excl_voltage %}
        {% if not deriv.is_voltage %}const double {% endif %}
        {{ deriv.lhs }} = {{ deriv.rhs }}; // {{ deriv.units }}
        {% endif %}
        {%- endfor %}
    }

    {% for deriv in y_derivatives %}
    rDY[{{ loop.index0 }}] = {{ deriv }};
    {%- endfor %}
}
```

Listing 2: Simplified Jinja2 template generating the `EvaluateYDerivatives` method.

```
void CellLuoRudy1991FromCellML::EvaluateYDerivatives(
    double var_chaste_interface__environment__time,
    const std::vector<double>& rY,
    std::vector<double>& rDY)
{
    // Inputs:
    // Time units: millisecond
    double var_chaste_interface__membrane__V =
        (mSetVoltageDerivativeToZero ? this->mFixedVoltage : rY[0]);
    // Units: millivolt; Initial value: -83.853
    double var_chaste_interface__intracellular_calcium_concentration__Cai = rY[1];
    // Units: millimolar; Initial value: 0.0002
    ...

    // Mathematics
    double d_dt_chaste_interface_var_membrane__V;
    const double var_slow_inward_current__E_si =
        7.7000000000000002
        - 13.028700000000001
        * log(var_chaste_interface__intracellular_calcium_concentration__Cai);
    const double var_slow_inward_current__i_si = ...
    ...

    if (mSetVoltageDerivativeToZero)
    {
        d_dt_chaste_interface_var_membrane__V = 0.0;
    }
    else
    {
        const double var_membrane__C = 1;
        const double var_membrane__I_stim =
            GetIntracellularAreaStimulus(var_chaste_interface__environment__time);
        ...
        d_dt_chaste_interface_var_membrane__V =
            -(var_background_current__i_b
              + var_fast_sodium_current__i_Na
              + var_membrane__I_stim
              + var_plateau_potassium_current__i_Kp
              + var_slow_inward_current__i_si
              + var_time_dependent_potassium_current__i_K
              + var_time_independent_potassium_current__i_K1)
            / var_membrane__C;
    }

    rDY[0] = d_dt_chaste_interface_var_membrane__V;
    rDY[1] =
        d_dt_chaste_interface_var_intracellular_calcium_concentration__Cai;
    ...
}
```

Listing 3: Shortened generated CPU implementation of `EvaluateYDerivatives` for the Luo–Rudy 1991 model, including the membrane voltage derivative from Equation 3.1.

3.2 GPU Integration into Chaste

To support GPU execution of cardiac cell models within Chaste, this work introduces a new abstraction, `CardiacCellCollection`, for evaluating multiple cells through batched ODE solves.

Efficient GPU execution requires uniform control flow (see Chapter 6), meaning that all cells within a collection must share the same cardiac cell model and numerical method. This is enforced by templating the collection on a concrete cardiac cell type, which exposes its associated GPU solver through a type-level interface, as illustrated below:

```
class CellLuoRudy1991FromCellML : public AbstractCardiacCell
{
public:
    template<typename ValueType>
    using SolverKernelLauncher =
        EulerKernelLauncher<CellLuoRudy1991FromCellML, ValueType>;
    ...
};
```

The `CardiacCellCollection` can then retrieve this launcher directly from the cell type:

```
template<typename CellType, typename ValueType>
class CardiacCellCollection {
public:
    using SolverKernelLauncher =
        typename CellType::template SolverKernelLauncher<ValueType>;
    ...
};
```

This allows the collection to invoke the GPU solve without any knowledge of the underlying numerical method or solver implementation:

```
template<typename CellType, typename ValueType>
std::vector<OdeSolution>
CardiacCellCollection<CellType, ValueType>::Compute(...) {
    ...
    SolverKernelLauncher::LaunchComputeOdeSolution(...);
    ...
}
```

Similar to the pre-existing CPU Forward Euler solver described earlier, the GPU Forward Euler implementation requires access to a model-specific derivative evaluation routine generated by `chaste-codegen`. Earlier, this was introduced through the generated `EvaluateYDerivatives` method provided for each cardiac cell model.

To support GPU execution, we introduce a corresponding device-compatible implementation:

```
template<typename ValueType>
__device__
static void EvaluateYDerivativesDevice(...);
```

The `Device` suffix distinguishes GPU-compatible implementations from their host-side equivalents. The rationale for introducing separate device implementations is discussed further in Section 4.1, while a generated GPU implementation of `EvaluateYDerivativesDevice` for the Luo–Rudy 1991 model is shown in Section 4.2.

The GPU Forward Euler solver can then invoke this method through the templated cell-type interface:

```
CellType::EvaluateYDerivativesDevice(...);
```

More advanced solvers, including Backward Euler and CVODE, require additional generated routines, as discussed in Chapter 2. The GPU equivalents of these routines follow the same naming convention. The resulting interface preserves the structure of the single-cell workflow shown in Figure 3.1, but evaluates a collection of cells through a single GPU-backed call. Listing 4 shows an example workflow using `CardiacCellCollection`, while Figure 3.2 shows the resulting voltage traces.

The `CardiacCellCollection` interface targets users such as cardiac modellers who are not typically familiar with GPU programming models, motivating a design consistent with existing Chaste workflows. The collection takes ownership of the cardiac cell objects using `std::unique_ptr` and synchronises their state $(\mathbf{y}, V)$ before and after GPU execution. Consequently, `CardiacCellCollection::Compute(...)` both performs the GPU solve and updates the underlying CPU-side cell states.

By keeping the CPU-side states updated, this design preserves compatibility with the existing Chaste serialisation infrastructure, including checkpoint saves and restart functionality, for workflows involving repeated batched solves. However, serialisation during active GPU execution is not currently supported and this compatibility has not yet been validated experimentally.

The implementation assumes that all cells within a collection share the same stimulus definition and membrane capacitance throughout a solve. These assumptions match typical Chaste workflows while simplifying the GPU implementation.

```
#include "CardiacCellCollection.hpp"
#include "EulerIvpOdeSolver.hpp"
#include "RegularStimulus.hpp"
#include "LuoRudy1991.hpp"

// Shared solver and stimulus
boost::shared_ptr<AbstractIvpOdeSolver> p_solver(new EulerIvpOdeSolver());

boost::shared_ptr<SimpleStimulus> p_stimulus(
    new SimpleStimulus(-25.5, 2.0, 100.0));

// Construct a collection of cells
std::vector<std::unique_ptr<CellLuoRudy1991FromCellML>> cells;

const unsigned num_cells = 32;
const double g_min = 0.22;
const double g_max = 0.34;

// Vary each cell's rapid delayed rectifier conductance (g_Kr)
for (unsigned i = 0; i < num_cells; ++i)
{
    const double alpha =
        static_cast<double>(i) / static_cast<double>(num_cells - 1);

    auto p_cell = std::make_unique<CellLuoRudy1991FromCellML>(
        p_solver,
        p_stimulus);

    p_cell->SetParameter(
        "membrane_rapid_delayed_rectifier_potassium_current_conductance",
        g_min + alpha * (g_max - g_min));

    cells.push_back(std::move(p_cell));
}

CardiacCellCollection<CellLuoRudy1991FromCellML, double> collection(
    std::move(cells));

CardiacCellCollection<CellLuoRudy1991FromCellML, double>::ComputeParams params;
params.dt = 0.01; params.tSamp = 0.01;

std::vector<OdeSolution> solutions = collection.Compute(0.0, 1000.0, params);

// Write the first solution to file
solutions[0].WriteToFile(
    "TestForwardEulerCells",
    "LuoRudy1991ForwardEulerGpuCollection",
    "ms",
    1,
    false,
    12);
```

Listing 4: GPU Forward Euler workflow evaluating 32 Luo–Rudy 1991 cells with varying delayed rectifier potassium conductance.

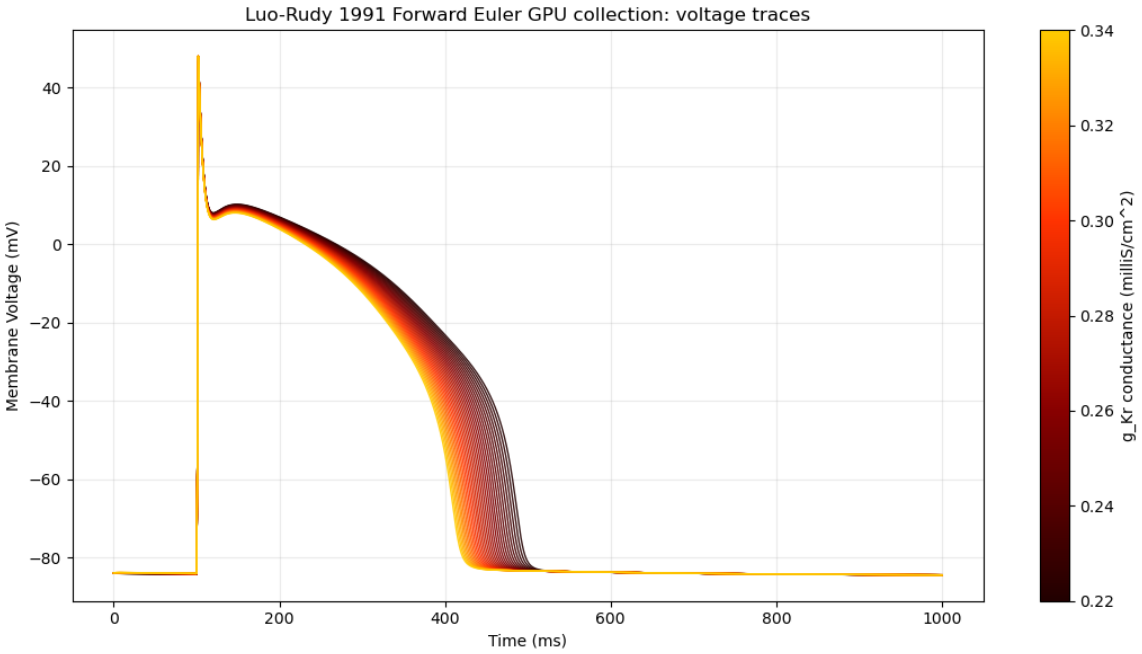


Figure 3.2: Voltage traces produced by the GPU workflow in Listing 4. Increasing delayed rectifier potassium conductance shortens the repolarisation duration as expected [16].

The collection is additionally templated on the underlying numerical type (`ValueType`), allowing execution in either single or double precision. This is particularly relevant for GPU execution given the hardware constraints discussed in Section 6.3.

3.3 Integration of GPU-accelerated CVODE

CVODE provides significantly higher performance than Chaste's built-in fixed-step solvers on CPU through its adaptive BDF integration strategy, making it an attractive target for GPU acceleration. However, incorporating the CUDA-enabled SUNDIALS infrastructure into Chaste presents several challenges. CVODE is configured for a fixed numerical precision at build time, which contrasts with the templated precision used in the generated GPU code and effectively restricts CVODE-based GPU support to double precision. More fundamentally, CVODE-based cell models follow a separate inheritance hierarchy. Chaste's built-in cardiac cell solvers operate on `std::vector<double>`, while CVODE cells operate on the SUNDIALS `N_Vector` type. In both cases, these types represent the cardiac cell state vector

$\mathbf{y}$ introduced earlier, but expose different interfaces and storage models. This difference prevents a unified abstraction (for example, methods such as `rGetStateVariables` return incompatible types). Combined with the distinct solver workflow required by CVODE's GPU interface, this makes integration into a shared abstraction difficult. For this work, the two approaches were therefore handled separately, with GPU-enabled CVODE cells implemented through a dedicated `CvodeCardiacCellCollection` alongside the `CardiacCellCollection` abstraction introduced for Chaste's built-in solvers.

3.4 Chaste Build System

Supporting these GPU execution paths required corresponding extensions to the Chaste build system so that CUDA sources, generated model code, and optional GPU-enabled CVODE components could be compiled alongside the existing CPU infrastructure. CellML model definitions used by the tests are fetched during CMake configuration, while `chaste-codegen` is invoked during the build process to generate the required source files prior to compilation. Chaste uses CMake [14] for build configuration and CxxTest [7] for its test suite. The `chaste-codegen` package is maintained separately from Chaste, with its templates and generators assembled through Python packaging rather than the Chaste CMake build system.

Figure 3.3 shows a simplified view of the source tree, highlighting the components relevant to the GPU extension.

```
Chaste/
|-- cmake/                 <- build macros and finder scripts
|-- global/                <- GPU abstraction macros and utility functions
|-- ode/                   <- CVODE solver integration
|-- heart/
|   |-- src/               <- collections and GPU solvers
|   |-- test/
|   |   `-- CMakeLists.txt <- CUDA test integration
|   `-- CMakeLists.txt     <- CUDA compilation and linking
`-- CMakeLists.txt         <- CUDA detection and build flags

chaste-codegen/
|-- chaste_codegen/        <- symbolic analysis and template population
|   `-- templates/         <- new Jinja2 templates for CUDA output
```

Figure 3.3: Simplified source tree highlighting components affected by the GPU extension.

CUDA support is introduced at the root `CMakeLists.txt`, where CUDA availability is detected and controlled via a user-facing option:

```
option(Chaste_ENABLE_CUDA "Enable CUDA support" ${CMAKE_CUDA_COMPILER})

if(Chaste_ENABLE_CUDA AND CMAKE_CUDA_COMPILER)
    enable_language(CUDA)
    find_package(CUDAToolkit REQUIRED)
    set(Chaste_USING_CUDA ON CACHE BOOL "" FORCE)
endif()

option(Chaste_ENABLE_CUDA_CVODE
    "Enable CUDA-accelerated CVODE support"
    OFF)
```

This allows CPU-only builds to be preserved even on systems with CUDA installed. CUDA-accelerated CVODE is treated as a separate, opt-in feature, reflecting its additional dependency on a recent SUNDIALS version built with CUDA support enabled, which is not part of the default SUNDIALS configuration.

Dependency handling is extended within the `cmake/` directory, where SUNDIALS is resolved using a legacy finder script. CPU components are always required, while CUDA-specific components are only requested when explicitly enabled:

```
find_package(SUNDIALS MODULE REQUIRED COMPONENTS
    sundials_cvode
    sundials_nvecserial)

if(Chaste_ENABLE_CUDA_CVODE AND Chaste_USING_CUDA)
    find_package(SUNDIALS MODULE REQUIRED COMPONENTS
        sundials_nveccuda
        sundials_sunmatrixcusparse
        sundials_sunlinsolcusolversp)
endif()
```

Within the `ode/` module, GPU CVODE functionality is isolated using a dedicated compile-time guard:

```
#ifdef CHASTE_CVODE_GPU
// CUDA CVODE implementation
#endif
```

Within the `heart` module, these top-level options are consumed by the local `CMakeLists.txt`. When CUDA is enabled, handwritten `.cu` sources implementing the GPU solvers and collection logic are collected from `heart/src`. Model-specific CUDA device code generated by `chaste-codegen` is emitted as headers in the build tree and compiled through inclusion into these translation units, requiring no additional build targets for generated models.

```
set(HEART_CUDA_SRCS "")
if(Chaste_USING_CUDA)
    file(GLOB_RECURSE HEART_CUDA_SRCS
         CONFIGURE_DEPENDS
         "${CMAKE_CURRENT_SOURCE_DIR}/src/*.cu"
endif()

if(Chaste_USING_CUDA)
    list(APPEND Chaste_heart_SOURCES ${HEART_CUDA_SRCS})
endif()

chaste_do_component(heart ${Chaste_heart_SOURCES})
```

This preserves the existing component structure by adding CUDA sources to the existing `chaste_heart` target rather than introducing a separate GPU library. CUDA-specific properties are configured and the appropriate toolkit libraries linked only when required:

```
if(Chaste_USING_CUDA AND TARGET chaste_heart)
    set_property(TARGET chaste_heart PROPERTY CUDA_STANDARD 17)

    target_link_libraries(chaste_heart PRIVATE ${Chaste_CUDA_LIBRARIES})

    if(Chaste_USING_CUDA_CVODE)
        target_link_libraries(chaste_heart PRIVATE
            ${Chaste_CUDA_CVODE_LIBRARIES})
    endif()
endif()
```

Common Chaste macros within `cmake/` were also updated to handle CUDA-aware behaviour consistently across components and tests. In particular, CUDA and CUDA-CVODE test packs are skipped unless the corresponding support is enabled:

```
foreach(type ${TestPackTypes})
    if(${type} STREQUAL "Cuda" AND NOT Chaste_USING_CUDA)
        continue()
    endif()

    if(${type} STREQUAL "CudaCvode" AND NOT Chaste_USING_CUDA_CVODE)
        continue()
    endif()

    chaste_add_test(${testTargetName}
                    "${CMAKE_CURRENT_SOURCE_DIR}/${filename}")
endforeach()
```

Finally, `heart/test/CMakeLists.txt` applies CUDA-specific linking and debug options only to executables containing CUDA sources, avoiding unnecessary CUDA dependencies for ordinary CPU tests.

Chaste tests are written using CxxTest, where developers define test suites in `.hpp` files. During the build, CMake generates C++ test runners from these headers. This creates a constraint for CUDA tests: CUDA syntax cannot be placed directly inside the CxxTest header, since the generated runner is not compiled by `nvcc`.

To handle this, CUDA tests use a small host-callable wrapper. The CxxTest suite remains ordinary C++ and calls a function declared in a standard header:

```
// TestCellCollectionBackwardEuler.hpp
#include <cxxtest/TestSuite.h>
#include "TestCellCollectionBackwardEulerWrapper.hpp"

class TestCellCollectionBackwardEuler : public CxxTest::TestSuite
{
public:
    void TestCellCollectionBackwardEulerCudaVsCpu()
    {
        RunCellCollectionBackwardEulerTest();
    }
};
```

The corresponding implementation is placed in a CUDA source file compiled by `nvcc`:

```
// TestCellCollectionBackwardEuler.cu
void RunCellCollectionBackwardEulerTest()
{
    TestCorrectnessCudaBackwardEuler_Impl();
}
```

This preserves compatibility with the standard CxxTest workflow while allowing the GPU implementation to be compiled separately using the CUDA toolchain. The similarity between the existing CPU workflow and the GPU collection workflow also allowed substantial reuse of existing test logic.

Chapter 4

Generated Device Code for Cardiac Cell Models

4.1 GPU-Compatible Generated Cell Models

As discussed in Section 3.2, supporting GPU execution within Chaste requires corresponding extensions to `chaste-codegen`. The CPU-side model-specific routines introduced in Chapter 2 must therefore also be generated in GPU-compatible form. Following the naming convention introduced earlier, device implementations are distinguished by appending `Device` to the routine name.

The generated GPU code closely mirrors the existing CPU implementation, allowing much of the translation to be performed mechanically. However, CUDA device code cannot directly access host-side functionality or state, requiring adaptation of several constructs present in the CPU-generated code:

- Utility functions defined within Chaste (e.g. `Signum`).
- Member access to cardiac cell objects (e.g. `mFixedVoltage`).
- Global configuration (e.g. `HeartConfig::Instance()->GetCapacitance()`).
- Stimulus evaluation via `GetIntracellularAreaStimulus(t)`.

The following sections describe how these CPU-oriented constructs are translated into GPU-compatible code during generation.

4.2 Code Generation for the Forward Euler Method

For the Forward Euler method, this produces the single required device routine, `EvaluateYDerivativesDevice`, shown for the Luo–Rudy 1991 model in Listing 5. The structure closely mirrors the pre-existing CPU-generated implementation shown in Listing 3. Combined with a Forward Euler solver, this routine alone is sufficient to integrate the model.

Following the code, several differences from the CPU implementation can be observed. First, the function is annotated with `__device__` (line 2), the CUDA syntax for defining functions that are callable from device code. Second, the function is templated on `ValueType`, with all numerical literals explicitly cast to this type (line 1 for template, and example of casting on line 25). The existing CPU-generated Chaste code assumes double precision throughout, which naturally aligns with C++ using `double` for unsuffixed floating-point literals. Without explicit casts, operations involving these literals would still be performed in double precision, independently of `ValueType`, preventing the single-precision speedups discussed in Chapter 6. Very small constants present in some CellML models may also underflow to zero when represented in single precision.

A more substantial difference lies in the function signature. Compared to the CPU equivalent, the generated device function takes additional arguments (lines 7–11), including `fixedVoltage`, `setVoltageDerivativeToZero`, and model parameters. These explicitly provide the model data previously accessed through host-side object state and global configuration. Although `capacitance` and `stimulus` are unused in this example, they appear in larger generated models and are therefore included in all generated functions to provide a uniform solver interface and simplify code generation. The appendix contains the complete generated Luo–Rudy 1991 implementation together with excerpts from the larger ToR–ORd model (Appendix A).

Supporting this interface requires a small set of device-side abstractions. The `stimulus` argument is represented by a device-compatible callable object (see Section 4.5.2), while the `ChasteDevice` namespace (see Section 4.5.1) provides wrappers around device-compatible mathematical functions, replacing calls such as `sin`, `abs`, and `Signum`.

Finally, some naming changes reflect the absence of object state in device code.

```
template<typename ValueType>
__device__
void CellLuoRudy1991FromCellML::EvaluateYDerivativesDevice(
    ValueType var_chaste_interface__environment__time,
    const ValueType y[8],
    ValueType dy[8],
    const ValueType setVoltageDerivativeToZero,
    const ValueType fixedVoltage,
    const ValueType parameters[3],
    const ValueType capacitance,
    const DeviceStimulusFunctor<ValueType> stimulus)
{
    // Inputs:
    // Time units: millisecond
    ValueType var_chaste_interface__membrane__V =
        (setVoltageDerivativeToZero ? fixedVoltage : y[0]);
    // Units: millivolt; Initial value: -83.853
    ValueType var_chaste_interface__intracellular_calcium_concentration__Cai =
        y[1];
    ...

    // Mathematics
    ValueType d_dt_chaste_interface_var_membrane__V;
    const ValueType var_slow_inward_current__E_si =
        static_cast<ValueType>(7.7000000000000002)
        - static_cast<ValueType>(13.028700000000001)
        * ChasteDevice::Log<ValueType>(...); // millivolt
    ...

    if (setVoltageDerivativeToZero)
    {
        d_dt_chaste_interface_var_membrane__V = static_cast<ValueType>(0.0);
    }
    else
    {
        ...
        const ValueType var_membrane__I_stim =
            stimulus(var_chaste_interface__environment__time);
        ...
        d_dt_chaste_interface_var_membrane__V =
            -( ... + var_membrane__I_stim + ... )
            / var_membrane__C;
    }

    dy[0] = d_dt_chaste_interface_var_membrane__V;
    dy[1] =
        d_dt_chaste_interface_var_intracellular_calcium_concentration__Cai;
    ...
}
```

Listing 5: Generated GPU implementation of `EvaluateYDerivativesDevice` for the Luo–Rudy 1991 model, closely matching the CPU implementation of `EvaluateYDerivatives` in Listing 3.

Member-style names such as `mFixedVoltage` become argument-style names such as `fixedVoltage`, while reference-style prefixes such as `rY` are removed (e.g. `y`). These changes improve readability while preserving the structure of the generated implementation.

4.3 Code Generation for the Backward Euler Method

In Section 2.3 we discussed the model-specific routines generated by `chaste-codegen` for the Backward Euler method within Chaste, namely `ComputeResidual`, `ComputeJacobian`, `UpdateTransmembranePotential` and `ComputeOneStepExceptVoltage`.

The GPU implementation generates corresponding device-compatible versions of these routines. Whilst `ComputeResidualDevice`, `ComputeJacobianDevice`, and `UpdateTransmembranePotentialDevice` correspond directly to their CPU counterparts, the functionality previously contained within `ComputeOneStepExceptVoltage` is decomposed into several smaller device routines.

The three routines `SolveClosedFormVarsDevice`, `FillInitialGuessDevice`, and `ScatterSolutionDevice` together implement the semi-implicit update strategy described in Section 2.3. `SolveClosedFormVarsDevice` computes the explicit voltage and closed-form gating updates from Equations 2.15 and 2.14, producing the precomputed state $\mathbf{u}^*_{n+1}$ from Equation 2.16. `FillInitialGuessDevice` then constructs the reduced nonlinear state $\mathbf{z}^{(0)}$ used to initialise the Newton iteration in Equation 2.19, while `ScatterSolutionDevice` writes the converged nonlinear solution $\mathbf{z}^{(k)}$ back into the full state vector $\mathbf{y}$.

Listing 6 shows the generated `SolveClosedFormVarsDevice` routine for the Luo–Rudy 1991 model which clearly computes updates of the form in Equation 2.14. The remaining generated device functions follow the same pattern and provide the other model-specific functions required by a GPU Backward Euler solver.

```
1  template<typename ValueType, typename ParameterAccessor>
2  DEVICE
3  void CellLuoRudy1991FromCellMLBackwardEuler::SolveClosedFormVarsDevice(
4      ValueType y[8],
5      const ValueType dt,
6      const ParameterAccessor parameters,
7      const bool setVoltageDerivativeToZero,
8      const ValueType fixedVoltage,
9      const ValueType capacitance)
10 {
11     ValueType var_chaste_interface__membrane__V =
12         (setVoltageDerivativeToZero ? fixedVoltage : y[0]);
13     // Units: millivolt; Initial value: -83.853
14     ...
15     ValueType var_chaste_interface__time_dependent_potassium_current_X_gate__X =
16         y[7];
17     // Units: dimensionless; Initial value: 0.16647703
18     ...
19     const ValueType var_time_dependent_potassium_current_X_gate__alpha_X =
20         static_cast<ValueType>(0.00050000000000000001)
21         * ChasteDevice::Exp<ValueType>(
22             static_cast<ValueType>(4.1500000000000004)
23             + static_cast<ValueType>(0.08300000000000004)
24               * var_chaste_interface__membrane__V)
25         / (1 + ChasteDevice::Exp<ValueType>(
26             static_cast<ValueType>(2.8500000000000001)
27             + static_cast<ValueType>(0.057000000000000002)
28               * var_chaste_interface__membrane__V));
29
30     const ValueType var_time_dependent_potassium_current_X_gate__beta_X =
31         static_cast<ValueType>(0.0012999999999999999)
32         * ChasteDevice::Exp<ValueType>(
33             static_cast<ValueType>(-1.2)
34             - static_cast<ValueType>(0.059999999999999998)
35               * var_chaste_interface__membrane__V)
36         / (1 + ChasteDevice::Exp<ValueType>(
37             static_cast<ValueType>(-0.80000000000000004)
38             - static_cast<ValueType>(0.040000000000000001)
39               * var_chaste_interface__membrane__V));
40     ...
41     y[7] =
42         (var_chaste_interface__time_dependent_potassium_current_X_gate__X
43          + var_time_dependent_potassium_current_X_gate__alpha_X * dt)
44         / (1.0
45            - (-var_time_dependent_potassium_current_X_gate__alpha_X
46               - var_time_dependent_potassium_current_X_gate__beta_X)
47              * dt);
48     ...
49 }
```

Listing 6: Generated `SolveClosedFormVarsDevice` fragment for the time-dependent potassium current `X` gate in the Luo–Rudy 1991 Backward Euler model. Lines 41-47 compute an update of the style in Equation 2.14.

4.4 Code Generation for the CVODE Solver

As discussed in Section 2.5, GPU execution with CVODE requires host-launched GPU routines evaluating the combined derivative system

$$\mathbf{F}(\mathbf{Y}) = \begin{pmatrix} \mathbf{f}_1(\mathbf{y}_1) \\ \mathbf{f}_2(\mathbf{y}_2) \\ \vdots \\ \mathbf{f}_N(\mathbf{y}_N) \end{pmatrix}, \tag{4.1}$$

together with the associated block-diagonal Jacobian

$$J = \frac{\partial \mathbf{F}}{\partial \mathbf{Y}} = \begin{pmatrix} J_1 & 0 & \cdots & 0 \\ 0 & J_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & J_N \end{pmatrix}. \tag{4.2}$$

Unlike the Forward Euler and Backward Euler implementations described previously, these routines must operate cooperatively across many uncoupled cell systems to construct the larger batched system expected by the GPU-enabled SUNDIALS interface.

The previously introduced `EvaluateYDerivativesDevice` routine already provides the per-cell right-hand side evaluations $\mathbf{f}_i(\mathbf{y}_i)$ required to construct the combined derivative system $\mathbf{F}(\mathbf{Y})$ in Equation 4.1. However, CVODE additionally requires analytic Jacobian evaluations during the Newton solves performed internally as part of the BDF integration process.

To support this, `chaste-codegen` is extended to generate a new device-compatible `EvaluateAnalyticJacobianDevice` routine for each cardiac cell model, following the same translation rules described previously. These generated routines compute the individual Jacobian blocks J_i from Equation 4.2. During execution, host-launched CUDA kernels invoke these routines for every cell in the batch and assemble the resulting per-cell Jacobian blocks into the larger block-diagonal structure required by the GPU-enabled SUNDIALS linear solver interface (Section 7.4). Listing 7 shows a simplified generated example.

In addition to the dense Jacobian representation described above, `chaste-codegen` is also extended to generate sparse Jacobian implementations through the routine `EvaluateSparseAnalyticJacobianDevice`. Unlike the dense `EvaluateAnalytic-JacobianDevice` routine, this implementation has no direct counterpart in the existing CPU code and required additional changes to the symbolic analysis performed during code generation, discussed later in Section 5.4.

```
template<typename ValueType>
__device__
void CellLuoRudy1991FromCellMLCvode::EvaluateAnalyticJacobianDevice(
    ValueType time,
    const ValueType y[8],
    ValueType dy[8],
    ValueType jacobian[8][8],
    ...)
{
    ValueType var_chaste_interface__membrane__V = y[0];
    ValueType var_chaste_interface__time_dependent_potassium_current_X_gate__X =
        y[7];

    ...

    // Matrix entries
    jacobian[7][0] =
        setVoltageDerivativeToZero ? static_cast<ValueType>(0.0)
        : (var_x22 * var_x23);

    jacobian[7][7] =
        -static_cast<ValueType>(0.0005000000000000001) * var_x91
        - static_cast<ValueType>(0.0012999999999999999) * var_x86;
}
```

Listing 7: Dense generated device Jacobian entries for the Luo–Rudy 1991 X-gate equation.

Sparse representations are particularly relevant for cardiac ionic models because many state variables interact only weakly. In particular, the gating equations from Equation 2.2 depend only on membrane voltage and the corresponding gating variable, producing Jacobian rows with only two nonzero entries. Consequently, cardiac cell Jacobians often exhibit substantial sparsity.

Rather than writing every entry of the per-cell Jacobian block J_i, the sparse implementation emits only structurally nonzero entries using the compressed sparse row (CSR) format illustrated in Figure 4.1. CSR stores the nonzero values consecutively in memory together with two index arrays: `row_ptr`, identifying where each matrix row begins, and `col_ind`, storing the corresponding column indices.

Both the dense and sparse generated routines construct the same block-diagonal system from Equation 4.2; they differ only in the storage representation used within each block. This correspondence can be seen in Listings 7 and 8.

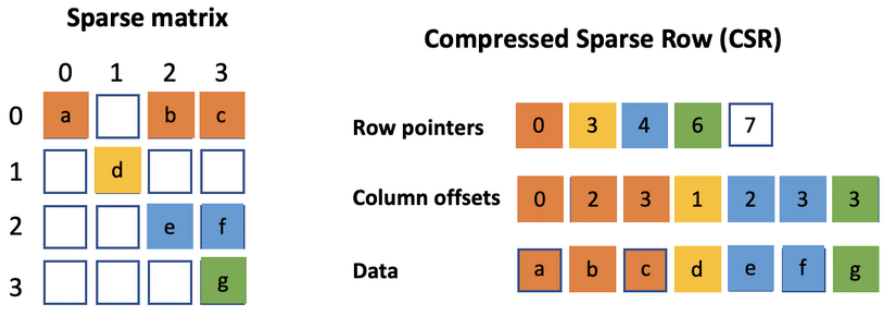


Figure 4.1: Example compressed sparse row (CSR) matrix representation. Reproduced from [13].

```
template<typename ValueType>
__device__
void CellLuoRudy1991FromCellMLCvode::EvaluateSparseAnalyticJacobianDevice(
    ValueType time,
    const ValueType y[8],
    ValueType dy[8],
    ValueType jacobian[24],
    ...)
{
    ValueType var_chaste_interface__membrane__V = y[0];
    ValueType var_chaste_interface__time_dependent_potassium_current_X_gate__X =
        y[7];
    ...

    // Matrix Entries
    jacobian[22] =
        setVoltageDerivativeToZero ? static_cast<ValueType>(0.0)
        : (var_x22 * var_x23);

    jacobian[23] =
        -static_cast<ValueType>(0.00050000000000000001) * var_x91
        - static_cast<ValueType>(0.0012999999999999999) * var_x86;
}
```

Listing 8: Sparse generated device Jacobian entries corresponding to Listing 7. Only structurally nonzero entries are written using the CSR format

4.5 Device-Side Support Utilities

4.5.1 ChasteDevice

Generated device functions rely on a shared utility namespace, `ChasteDevice`, which provides device-compatible replacements for mathematical functions used by the CPU code generator. These wrappers are templated on `ValueType`, ensuring use of the correct precision-specific device instruction. Listing 9 shows a simplified subset of these wrappers. Standard mathematical calls are replaced mechanically with their `ChasteDevice` equivalents; for example, `sin(x)` becomes `ChasteDevice::Sin<ValueType>(x)`. When instantiated with `ValueType = float`, this resolves to `sinf(x)`, while `ValueType = double` resolves to `sin(x)`.

```
namespace ChasteDevice {

inline constexpr double E  = 2.71828182845904523536;
inline constexpr double Pi = 3.14159265358979323846;

#define CUDA_MATH_1ARG(FuncName, FloatFunc, DoubleFunc)      \
template<typename ValueType, typename Arg>                   \
__device__ __forceinline__                                   \
ValueType FuncName(Arg x)                                    \
{                                                            \
    if constexpr (std::is_same_v<ValueType, float>)          \
        return FloatFunc(static_cast<float>(x));             \
    else                                                     \
        return DoubleFunc(static_cast<double>(x));           \
}

CUDA_MATH_1ARG(Sin,  sinf,  sin)
CUDA_MATH_1ARG(Exp,  expf,  exp)
CUDA_MATH_1ARG(Log,  logf,  log)
CUDA_MATH_1ARG(Sqrt, sqrtf, sqrt)

template<typename T>
__device__ __forceinline__
T Sign(T x)
{
    return static_cast<T>((T(0) < x) - (x < T(0)));
}
#undef CUDA_MATH_1ARG
}
```

Listing 9: Simplified subset of the `ChasteDevice` wrappers used by generated device code.

4.5.2 DeviceStimulusFunctor

Stimulus evaluation requires a broader change than the mathematical wrappers described above. In the CPU implementation, stimuli are represented as C++ objects evaluated through calls such as `GetIntracellularAreaStimulus(t)`. This cannot be used directly in generated device code, since GPU device functions cannot access host-side objects or virtual class hierarchies.

To address this, each supported CPU stimulus implements a method, `MakeStimulusDescriptor`, which emits a fixed-size representation of the stimulus suitable for transfer to the GPU. A cardiac cell may then expose its stimuli through:

```
std::vector<StimulusDescriptor>
GetIntracellularStimulusDescriptor() const
{
    if (!mpIntracellularStimulus)
    {
        return { StimulusDescriptor() };
    }

    return mpIntracellularStimulus->MakeStimulusDescriptor();
}
```

Each `StimulusDescriptor` stores the parameters required for GPU-side stimulus evaluation:

```
enum StimKind : int {
    STIM_ZERO = 0,
    STIM_SIMPLE = 1,
    STIM_REGULAR = 2,
    STIM_REGULAR_ZERO_NET_CHARGE = 3
};

template<typename ValueType>
struct StimulusDescriptorT {
    int kind;
    int n_params;
    ValueType params[8];
    ValueType eps;
};
```

For example, a regular periodic stimulus emits a descriptor containing its magnitude, duration, period, and timing information. These descriptors are then used to construct a GPU-compatible evaluator dispatching on the stimulus type:

```cpp
template<typename ValueType>
__device__
ValueType EvalDeviceStimulus(
    const StimulusDescriptorT<ValueType>& d,
    ValueType time)
{
    switch (d.kind)
    {
    case STIM_SIMPLE:
        ...
    case STIM_REGULAR: {
        ValueType mag    = d.params[0];
        // Extend duration slightly to avoid missing the final timestep
        ValueType dur    = d.params[1] + d.params[2] * d.eps;
        ValueType period = d.params[2];
        ValueType start  = d.params[3];
        ValueType stop   = d.params[4];
        if (time < start || time > stop) {
            return static_cast<ValueType>(0);
        }
        ValueType beat = fmod(time - start, period);
        return (beat >= 0 && beat <= dur) ? mag : static_cast<ValueType>(0);
    }
    case STIM_REGULAR_ZERO_NET_CHARGE:
        ...
    default:
        return static_cast<ValueType>(0);
    }
}

template<typename ValueType, int MAX_STIMS=16>
struct DeviceStimulusFunctor {

    int count;
    StimulusDescriptorT<ValueType> descs[MAX_STIMS];

    __device__
    ValueType operator()(ValueType t) const
    {
        ValueType sum = ValueType{0};
        for (int i = 0; i < count; ++i)
        {
            sum += EvalDeviceStimulus<ValueType>(descs[i], t);
        }
        return sum;
    }
};
```

Listing 10: Simplified GPU stimulus evaluation infrastructure.

A fixed double-precision alias is introduced to avoid templating the existing CPU stimulus hierarchy:

```
using StimulusDescriptor = StimulusDescriptorT<double>;
```

The CPU stimulus hierarchy therefore operates on a fixed double-precision descriptor type, whilst generated GPU code may instantiate the same descriptors in either single or double precision.

These descriptors are retrieved from the cardiac cell and converted to the required precision when constructing the device-side stimulus functor:

```
std::vector<StimulusDescriptor> descs =
    cell->GetIntracellularStimulusDescriptor();

DeviceStimulusFunctor<ValueType> dev_stim(descs);
```

This provides a GPU-compatible stimulus infrastructure for generated cardiac cell device functions. We consider two representative stimulus types: the regular stimulus, which applies a periodic rectangular current pulse, and the zero-net-charge stimulus, which applies equal and opposite positive and negative phases such that the total injected charge is approximately zero. Figure 4.2 shows example waveforms for a single pulse of each stimulus type.

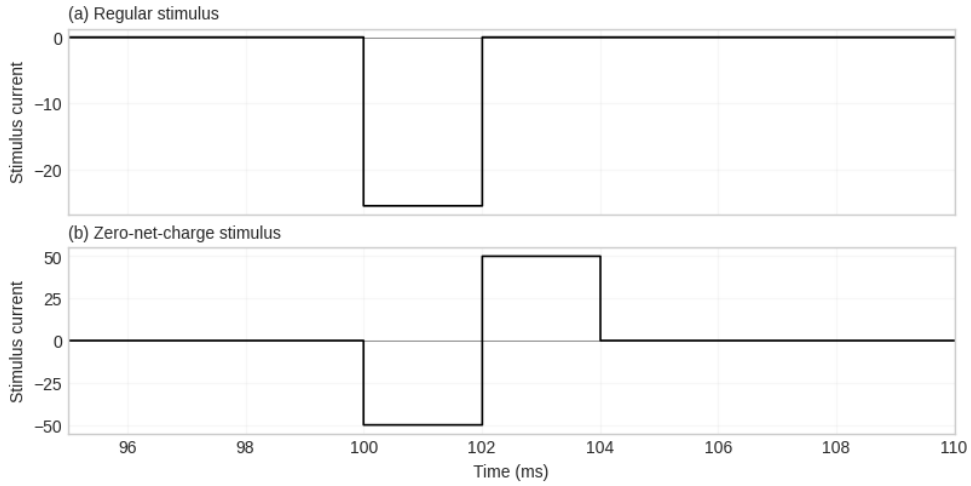


Figure 4.2: Example regular and zero-net-charge stimulus waveforms.

These stimuli were applied to GPU simulations of the Luo–Rudy 1991 model using the `CardiacCellCollection` infrastructure described previously. Figure 4.3 shows the resulting GPU-generated action potentials for regular, zero-net-charge, and combined multi-stimulus pacing configurations. Quantitative CPU–GPU validation is presented later in Chapter 8.

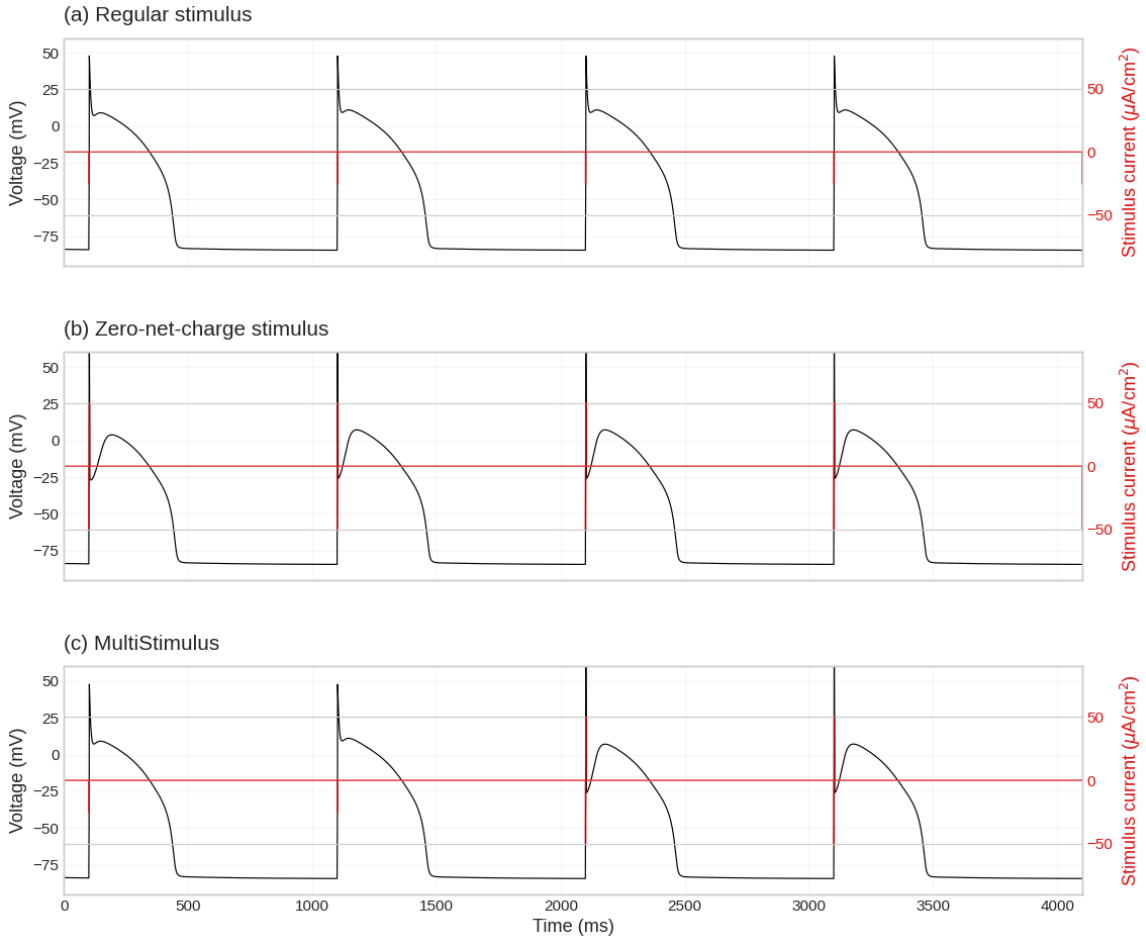


Figure 4.3: GPU-generated Luo–Rudy 1991 action potentials for regular, zero-net-charge, and combined multi-stimulus pacing configurations.

Whilst the generated stimulus infrastructure supports the configurations above, it introduces additional storage overhead. Since the number of stimuli must be known at compile time, a fixed upper bound is used (`MAX_STIMS = 16`), which may increase register pressure or force values into local memory during kernel execution.

An alternative approach, used in MonoAlg3D [2], precomputes stimulus values on the host and passes them as a device array, reducing device-side evaluation complexity at the cost of additional storage and memory transfers.

Together, these mechanisms allow existing CPU-generated cardiac cell routines to be translated into GPU-compatible implementations whilst largely preserving their symbolic structure. Since the generated code relies only on standard C++ and simple CUDA/HIP-compatible device features [18, 1], and therefore could, in principle,

target both NVIDIA and AMD GPU toolchains.

These generated routines provide the model-specific functionality required by the GPU solver implementations described in Chapter 7.

Chapter 5

Automatic Code Generation for Cardiac Cell Models

5.1 Integrating Device Code into Generated Cell Classes

Having introduced the existing `chaste-codegen` pipeline in Section 3.1, and the generated device functions required for GPU execution in Chapter 4, we now turn to the modifications required to generate these device-compatible implementations. We begin by examining the Jinja2 [20] templates responsible for emitting the generated code, which provide a concrete view of how the device functions are integrated into the cardiac cell classes.

Jinja2 templates interleave ordinary source code with placeholder variables and control structures. Double-brace expressions are replaced with strings generated during code generation, while Jinja control statements determine which code is emitted. Templates may also include other templates, allowing shared code fragments to be reused.

In the original CPU code generation, these templates are used to assemble the standard C++ header and implementation files corresponding to a concrete cardiac cell model. Our GPU extension augments this process by declaring additional device-compatible methods in the generated header, and by emitting their implementations in a separate, device-specific header. Listing 11 shows the relevant portion of the Forward Euler header template responsible for declaring these methods. Highlighted lines correspond to modifications introduced for GPU support, while unhighlighted lines show the surrounding pre-existing template structure. Green highlighting de-

notes newly added code, while red highlighting indicates CPU-specific logic removed or generalised during the GPU extension.

```
{% include "Shared/hpp/header_comments" %}
{% include "Shared/hpp/includes" %}
#include "{{base_class}}.hpp"
{% if generate_kernels -%}
#if USING_DEVICE_COMPILER
#include "EulerKernels.hpp"
#endif
{%- endif %}
{% include "Shared/hpp/class_declaration" %}
{% if generate_kernels -%}
#if USING_DEVICE_COMPILER
    template<typename ValueType> using SolverKernelLauncher =
        EulerKernelLauncher<{{class_name}}, ValueType>;
    static constexpr unsigned TOTAL_SIZE = {{ state_vars|length }}u;
#endif
{%- endif %}
{% include "Shared/hpp/DefaultStimulus_IntracellularCalciumConcentration" %}
    {{class_name}}(boost::shared_ptr<AbstractIvpOdeSolver> pSolver, boost::
    shared_ptr<AbstractStimulusFunction> pIntracellularStimulus);
{% include "Shared/hpp/destructor_verify_state_variables_GetIIonic" %}
    void EvaluateYDerivatives(...);
{% if generate_kernels -%}
#if USING_DEVICE_COMPILER
    template<typename ValueType, typename ParameterAccessor>
    __device__
    static void EvaluateYDerivativesDevice(...);
#endif
{%- endif %}
{% include "Shared/hpp/ComputeDerivedQuantities" %}
{% include "Shared/hpp/CHASTE_CLASS_EXPORT" %}

{% if generate_kernels -%}
#if USING_DEVICE_COMPILER
#include "{{ file_name }}Kernels.hpp"
#endif
{%- endif %}
```

Listing 11: Forward Euler header template with GPU-specific additions highlighted.

This template extends the existing CPU header by conditionally introducing device-specific declarations whilst preserving the surrounding class structure and CPU-facing interface. The overall structure of the class remains unchanged: the concrete cardiac cell still derives from the appropriate base class and retains its CPU-facing methods.

The solver header is included only when both `generate_kernels` and `USING_DEVICE_COMPILER` are enabled. `generate_kernels` is set by the `chaste-codegen` invocation

and controls whether GPU code is emitted, while `USING_DEVICE_COMPILER` is defined only when compiling with `nvcc`. Together, these guards prevent CUDA-specific constructs from being seen by a standard C++ compiler and allow the generated cardiac cell class to remain valid in both CPU-only and GPU-enabled builds.

Within the class, a new template alias, `SolverKernelLauncher`, associates the concrete cardiac cell type with its corresponding GPU solver at compile time. As described in Section 3.2, this allows `CardiacCellCollection` to invoke the appropriate solver through a type-level interface. The additional constant `TOTAL_SIZE` provides compile-time information about the state layout required by the solver.

The existing CPU method declaration, `EvaluateYDerivatives`, is retained unchanged. Under the same guards, the generated device method `EvaluateYDerivativesDevice` is additionally declared. Its implementation is emitted into the separate header `{{file_name}}Kernels.hpp`, included after the class declaration. Although these headers are named `*Kernels.hpp`, the generated functions are `__device__` routines rather than CUDA `__global__` kernels; "kernel" is used here in the broader sense of GPU-specific generated code.

The same pattern is used for the Backward Euler and CVODE-generated cell classes. Their headers declare the additional device functions described in Sections 4.3 and 4.4, provide the required compile-time state information, and include the corresponding implementation headers. Backward Euler similarly exposes a `SolverKernelLauncher` alias for use with `CardiacCellCollection`, while CVODE execution is handled separately through `CvodeCardiacCellCollection`.

5.2 Device Code Generation for Model-Specific Functions

For Forward Euler, the implementation header is compact because only `EvaluateYDerivativesDevice` is generated. Its structure is:

```
{% include "Shared/kernels/header_comments" %}
{% include "Shared/kernels/includes" %}
{% include "Shared/kernels/EvaluateYDerivativesDevice" -%}
{% include "Shared/kernels/footer" %}
```

The included templates provide common boilerplate shared across generated models. For example, the `header_comments` template emits metadata describing the originating CellML model, code generation configuration, and generation timestamp:

```
//! @file
//!
//! This source file was generated from CellML by chaste_codegen version 0.10.6
//!
//! Model: luo_rudy_1991
//!
//! Processed by chaste_codegen: https://github.com/ModellingWebLab/chaste-codegen
//!     (translator: chaste_codegen, model type: BackwardEuler)
//! on 2026-05-07 10:04:33
//!
//! <autogenerated>
```

Backward Euler and CVODE follow the same structure but require more generated device methods. Their implementation headers are therefore decomposed further, with separate sub-templates for residuals, Jacobians, voltage updates, and related routines.

Listing 12 shows the template used to generate the Forward Euler device right-hand-side routine. It produces the function in Listing 5 and closely mirrors the CPU template in Listing 2. The primary differences are device annotations, additional arguments, a templated `ValueType` argument, and GPU-compatible template variables.

```
template<typename ValueType>
__device__
void {{class_name}}::EvaluateYDerivativesDevice(
    ValueType {{free_variable.var_name}},
    const ValueType y[{{state_vars|length}}],
    ValueType dy[{{state_vars|length}}],
    const bool setVoltageDerivativeToZero,
    const ValueType fixedVoltage,
    const ValueType parameters[{{modifiable_parameters|length}}],
    const ValueType capacitance,
    const DeviceStimulusFunctor<ValueType> stimulus)
{
{% for state_var in state_vars %}
    ValueType {{state_var.var}} = {{state_var.rY_lookup_device}};
{% endfor %}

{% for deriv in y_derivative_equations_device %}
    const ValueType {{deriv.lhs}} = {{deriv.rhs}};
{% endfor %}

{% for state_var in state_vars %}
    dy[{{loop.index0}}] = {{state_var.derivative}};
{% endfor %}
}
```

Listing 12: Forward Euler `EvaluateYDerivativesDevice` template.

The remaining model-specific device functions required by the Backward Euler and CVODE solvers follow the same general pattern. Existing CPU templates are adapted into device-compatible forms by extending their signatures, introducing device annotations, templating on `ValueType`, and replacing their template variables.

5.3 Device-Specific Expression Generation

Whilst the new templates provide much of the required structure for generated CUDA device functions, the expressions inserted into the template variables must also be device-compatible. The existing CPU pipeline already constructs symbolic expressions for quantities such as derivatives, residuals, and Jacobians. GPU support therefore reuses these symbolic expressions, but modifies the object responsible for converting them into C++ strings, the `ChastePrinter`. Whilst the `ChastePrinter` is responsible for converting most symbolic expressions to strings, some quantities are inserted through formatting logic outside the printer itself. To handle these cases, we maintain separate dictionaries containing the CPU and GPU naming schemes for quantities such as the state vector, parameter storage, and fixed-voltage handling. Device code generation is then performed by temporarily switching both the active printer and these naming dictionaries using the following context manager:

```
@contextmanager
def _device_mode(self):
    original_state = getattr(self, '_is_generating_device_code', False)
    original_printer = getattr(self, '_printer', None)
    original_names = getattr(self, '_codegen_names', None)

    self._is_generating_device_code = True
    self._codegen_names = self._gpu_names   # Switch to GPU naming dictionary
    self._printer = self._device_printer    # Switch to GPU printer
    try:
        yield
    finally:
        self._is_generating_device_code = original_state
        self._codegen_names = original_names
        self._printer = original_printer

def _in_device_mode(self, func, *args, **kwargs):
    # e.g. self._in_device_mode(self._format_some_quantity_as_string,
    #                           self._quantity_to_format)
    with self._device_mode():
        return func(*args, **kwargs)
```

Within the `with` block, formatting routines execute with the device printer and naming scheme temporarily active, after which the CPU configuration is restored. This allows the same symbolic expressions to be emitted in both CPU and device forms during construction of the shared template-variable dictionary shown in Listing 13:

```
self._vars_for_template = \
            {'base_class': '',
             'model_type': '',
             # indicate how to declare state vars and values
             'vector_decl': 'std::vector<double>&',
             'converter_version': cg.__version__,
             'model_name': self._model.name,
             'file_name': self.file_name,
             'class_name': kwargs.get('class_name', 'ModelFromCellMl'),
             'header_ext': kwargs.get('header_ext', '.hpp'),
             'dynamically_loadable': kwargs.get('dynamically_loadable', False),
             'use_model_factory': kwargs.get('use_model_factory', False),
             'cellml_base': kwargs.get('cellml_base', ''),
             'modifiers': self._format_modifiers(),
             'generation_date': TIME_STAMP,
             'use_get_intracellular_calcium_concentration': self._model.
                cytosolic_calcium_concentration_var in self._model.state_vars,
             'membrane_voltage_index': MEMBRANE_VOLTAGE_INDEX,
             'cytosolic_calcium_concentration_index': self._state_vars.index(
                self._model.cytosolic_calcium_concentration_var)
                if self._model.cytosolic_calcium_concentration_var in self._model.
                    state_vars
                else CYTOSOLIC_CALCIUM_CONCENTRATION_INDEX,
             'modifiable_parameters': self._format_modifiable_parameters(),
             'state_vars': self._formatted_state_vars,
             'use_verify_state_variables': self._use_verify_state_variables,
             'default_stimulus_equations': self._format_default_stimulus(),
             'ionic_vars': self._format_ionic_vars(),
             'y_derivatives': self._format_y_derivatives(),
             'y_derivative_equations': self._format_derivative_equations(self.
                _derivative_equations),
             'y_derivative_equations_device': self._in_device_mode(self.
                _format_derivative_equations, self._derivative_equations),
             'free_variable': self._format_free_variable(),
             'ode_system_information': self._format_system_info(),
             'named_attributes': self._format_named_attributes(),
             'derived_quantities': self._format_derived_quant(),
             'derived_quantity_equations': self._format_derived_quant_eqs(),
             'generate_kernels': self.generate_kernels}
```

Listing 13: Shared template-variable dictionary populated by `ChasteModel` during code generation. Highlighted lines indicate GPU-specific additions.

The formatting routines themselves are unchanged, but delegate expression printing through the currently active printer:

```
def format_derivative_equation(self, eq, modifiers_with_defining_eqs):
    return {
        'lhs': self._printer.doprint(eq.lhs),
        'rhs': self._print_rhs_with_modifiers(...),
        'units': self._model.units.format(...),
        ...
    }
```

Consequently, switching the printer changes the emitted syntax while preserving the underlying symbolic expressions. These generated device-compatible variables can then be substituted directly into the templates:

```
{% for deriv in y_derivative_equations_device %}
const ValueType {{deriv.lhs}} = {{deriv.rhs}}; // {{deriv.units}}
{% endfor %}
```

The device-specific printer extends the existing CPU printer with replacements required for CUDA-compatible code generation:

```
class ChasteDevicePrinter(ChastePrinter):

    _function_names = {
        'exp': 'ChasteDevice::Exp',
        'log': 'ChasteDevice::Log',
        'sqrt': 'ChasteDevice::Sqrt',
        'sin': 'ChasteDevice::Sin',
        'cos': 'ChasteDevice::Cos',
    }

    _literal_names = {
        'pi': 'static_cast<ValueType>(ChasteDevice::Pi)',
    }

    def _print_Function(self, expr):
        name = expr.func.__name__

        if name == 'GetIntracellularAreaStimulus':
            return f'stimulus({self._print(expr.args[0])})'

        if name == 'HeartConfig::Instance()->GetCapacitance':
            return 'capacitance'

        if name in self._function_names:
            args = ', '.join(self._print(arg) for arg in expr.args)

            return (
                f'{self._function_names[name]}'
                f'<ValueType>({args})'
            )

        return super()._print_Function(expr)
```

This printer redirects mathematical operations to the `ChasteDevice` wrappers described in Section 4.5.1, while also replacing host-side constructs such as stimulus and capacitance accessors with the corresponding device-side function arguments.

Some CPU-specific naming behaviour was also embedded directly within formatting functions and therefore required additional abstraction. In particular, quantities such as the state vector, parameter storage, and fixed-voltage handling previously relied on hard-coded member access which would be invalid within generated device functions. To address this, separate naming dictionaries are maintained for CPU and GPU code generation:

```
self._cpu_names = {
    'state': 'rY',
    'parameters': 'mParameters',
    'fixed_voltage': 'this->mFixedVoltage',
    'set_voltage_derivative_to_zero':
        'mSetVoltageDerivativeToZero',
}

self._gpu_names = {
    'state': 'y',
    'parameters': 'parameters',
    'fixed_voltage': 'fixedVoltage',
    'set_voltage_derivative_to_zero':
        'setVoltageDerivativeToZero',
}

self._codegen_names = self._cpu_names
```

Formatting routines can then reference these abstract names without embedding target-specific logic directly:

```
def _format_rY_lookup(self, index, var, use_modifier=True):
        """ Formatting of rY lookup for printing"""
        entry = self._format_rY_entry(index)
        if use_modifier and var in self._modifiers:
            entry = self._format_modifier(var) +\
                '->Calc(' + entry + ', ' + self._printer.doprint(self._model.
                    time_variable) + ')'
        if var == self._model.membrane_voltage_var:
            entry = '(mSetVoltageDerivativeToZero ? this->mFixedVoltage : ' +
                    entry + ')'
            fixed_voltage = self._codegen_names['fixed_voltage']
            set_voltage_derivative_to_zero = self._codegen_names[
                'set_voltage_derivative_to_zero']
            entry = '(' + set_voltage_derivative_to_zero + ' ? ' + fixed_voltage +
            ' : ' + entry + ')'
        return entry
```

The underlying array access is emitted separately:

```
def _format_rY_entry(self, index):
    """Formatting of rY entry for printing"""
    return 'rY[' + str(index) + ']'
    return self._codegen_names['state'] + '[' + str(index) + ']'
```

Notably, the modifier access path using `->Calc(...)` remains unchanged in the generated device code. Chaste and `chaste-codegen` support a wide range of experimental protocols and modifier types through this interface, many of which depend on host-side infrastructure that is not directly portable to device execution. Replicating the full modifier system on the GPU was beyond the scope of this project. Instead, GPU support focuses on the generated cardiac cell computations required for large-scale ensembles of independent single-cell simulations with varying parameter sets, while preserving the existing CPU-oriented code generation pipeline.

5.4 CVODE Device Code Generation

Code generation for the CVODE solver follows the same general pattern as Forward and Backward Euler as already described. However, supporting the sparse Jacobian representation introduced in Section 4.4 required additional code generation logic.

In the original implementation, when analytic Jacobians are requested, the symbolic Jacobian is constructed using `get_jacobian` and printed once for insertion into the CPU templates. For GPU support, this process is extended by evaluating the same symbolic expressions under `_device_mode`, producing a second set of template variables containing device-compatible code:

```
if self._use_analytic_jacobian:
    ...
    jac_eqs, jac_entries = self._print_jacobian()
    self._vars_for_template['jacobian_equations'] = jac_eqs
    self._vars_for_template['jacobian_entries'] = jac_entries
    jac_eqs_dev, jac_entries_dev = self._in_device_mode(self._print_jacobian)
    self._vars_for_template['jacobian_equations_device'] = jac_eqs_dev
    self._vars_for_template['jacobian_entries_device'] = jac_entries_dev
```

As with the Forward Euler derivative generation, this reuses the same symbolic Jacobian structure whilst emitting both CPU and device-compatible forms.

In addition to this, the generated non-zero Jacobian entries are sorted and used to construct the sparse structures required by the generated CVODE device implementation:

```
if self._use_analytic_jacobian:
    ...
    sorted_jac_entries = sorted(
        jac_entries_dev,
        key=lambda entry: (entry['i'], entry['j'])
    )

    row_counts = [0] * n_vars
    for entry in sorted_jac_entries:
        row_counts[entry['i']] += 1

    row_ptr = [0]
    for count in row_counts:
        row_ptr.append(row_ptr[-1] + count)

    col_ind = [entry['j'] for entry in sorted_jac_entries]

    self._vars_for_template['sparse_jacobian_entries'] = sorted_jac_entries
    self._vars_for_template['sparse_rowptr'] = row_ptr
    self._vars_for_template['sparse_colind'] = col_ind
    # Number of non-zero entries
    self._vars_for_template['sparse_nnz'] = len(sorted_jac_entries)
```

These arrays (`row_ptr`, `col_ind`) and the corresponding non-zero entries are passed to the templates via `_vars_for_template`. The generated cell headers then expose these structures as static data which can be consumed by the CVODE GPU solver to configure the Jacobian sparsity pattern.

If analytic Jacobians are not requested, the corresponding template variables are populated with empty structures, preserving the existing behaviour.

The sparse Jacobian generation therefore extends the existing symbolic code generation pipeline rather than introducing a separate implementation path.

Chapter 6

CUDA Execution Model for Cardiac Cell Simulation

6.1 Thread Execution Model

The previous sections described the generation of device-compatible cardiac cell methods. In the existing Chaste workflow illustrated in Figure 3.1, the generated methods for CPU are consumed by solver objects which perform the numerical solve. Before describing the GPU equivalents of these solver implementations in the following chapter, it is useful to introduce the small part of the CUDA execution model required here.

A CUDA kernel is executed by a large number of lightweight threads. For the purposes of programming, CUDA organises these threads into thread blocks, which are themselves grouped into a grid. This grid is a software execution model used to describe how work is launched, rather than a physical hardware layout. At runtime, thread blocks are scheduled dynamically onto the GPU's Streaming Multiprocessors (SMs), which are the hardware execution units responsible for running groups of threads concurrently, as illustrated in Figure 6.1.

Each thread has an index within its block (`threadIdx.x`), and each block has an index within the grid (`blockIdx.x`). The number of threads per block is given by `blockDim.x`.

Threads are executed in groups of 32 known as warps. All threads within a warp are executed in lockstep by the GPU hardware, meaning that they follow the same instruction stream at any given time. For this reason, threads are typically launched in multiples of 32, so that warps are fully utilised.

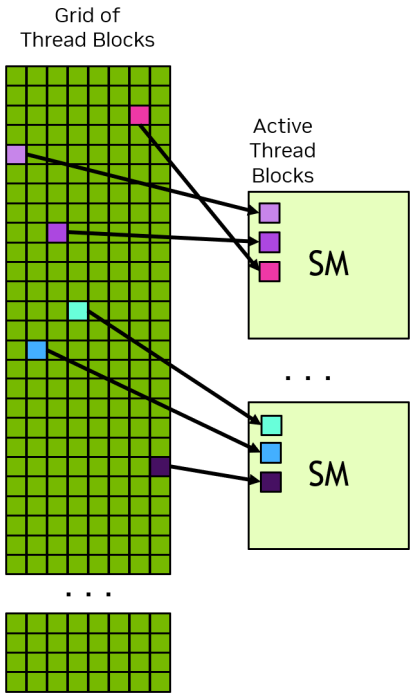


Figure 6.1: CUDA execution model showing thread blocks from a logical grid being scheduled dynamically onto Streaming Multiprocessors (SMs) [18].

For example, a kernel launch of the form

```
// Threads per block
int block_size = 128;
// Number of blocks
int grid_size = (number_of_cells + block_size - 1) / block_size;

MyKernel<<<grid_size, block_size>>>(...);
```

launches `grid_size * block_size` threads in total, which is the smallest multiple of `block_size` greater than or equal to `number_of_cells`. As a result, slightly more threads may be launched than required, but this aligns with the hardware execution model. Threads with indices beyond `number_of_cells` therefore simply exit early.

Within the kernel, each thread computes a global index as

```
int cell_idx = blockIdx.x * blockDim.x + threadIdx.x;
if (cell_idx >= number_of_cells) return;
```

In this implementation, the global thread index maps directly to a cell index, with each thread advancing one cardiac cell independently. This sequential, thread-local processing relies on the large number of cells to provide parallel saturation. To mitigate

warp divergence, where conditional branching serialises execution paths within a 32-thread warp, the `CardiacCellCollection` groups identical cell models and numerical methods together to ensure uniform code paths and maximise hardware throughput.

6.2 Memory Model

CUDA exposes a hierarchical memory model, illustrated in Figure 6.2. Each thread primarily uses registers for fast, on-chip storage, while the GPU device DRAM provides large-capacity memory accessible across all threads. Accesses to device memory are serviced by a hardware-managed cache hierarchy, consisting of an L1 cache local to each Streaming Multiprocessor (SM) and a larger L2 cache shared across the device.

In CUDA terminology, "global memory" can refer both to memory with device-wide visibility and, more informally, to storage in device DRAM. In this work, references to global memory generally mean arrays that are visible to all threads and stored in device memory. By contrast, thread-local variables are private to a single thread and are usually held in registers, although larger implicit models may require some values to spill into device memory. Performance is strongly influenced by memory access patterns: contiguous accesses across a warp can be coalesced into fewer memory transactions, whereas irregular accesses reduce bandwidth utilisation.

CUDA also provides programmer-managed shared memory accessible to all threads within a block. Shared memory is not used explicitly in this work because the dominant data structures are thread-local and exhibit limited reuse across threads or timesteps. Consequently, introducing it would add coordination overhead while reducing available L1 cache capacity. Future optimisations could, however, utilise shared memory to cache stimulus values or lookup-table-based model evaluations.

Nevertheless, there are scenarios where shared memory could become useful. As discussed in Section 4.5.2, stimulus values could be precomputed and stored in device arrays, as in MonoAlg3D [2], potentially enabling reuse across timesteps. Similarly, Chaste supports optimised CPU models which evaluate expensive expressions via lookup tables; a GPU implementation of this approach could cache portions of these tables in shared memory to reduce device memory traffic. These optimisations were not implemented here, but represent promising directions for future work.

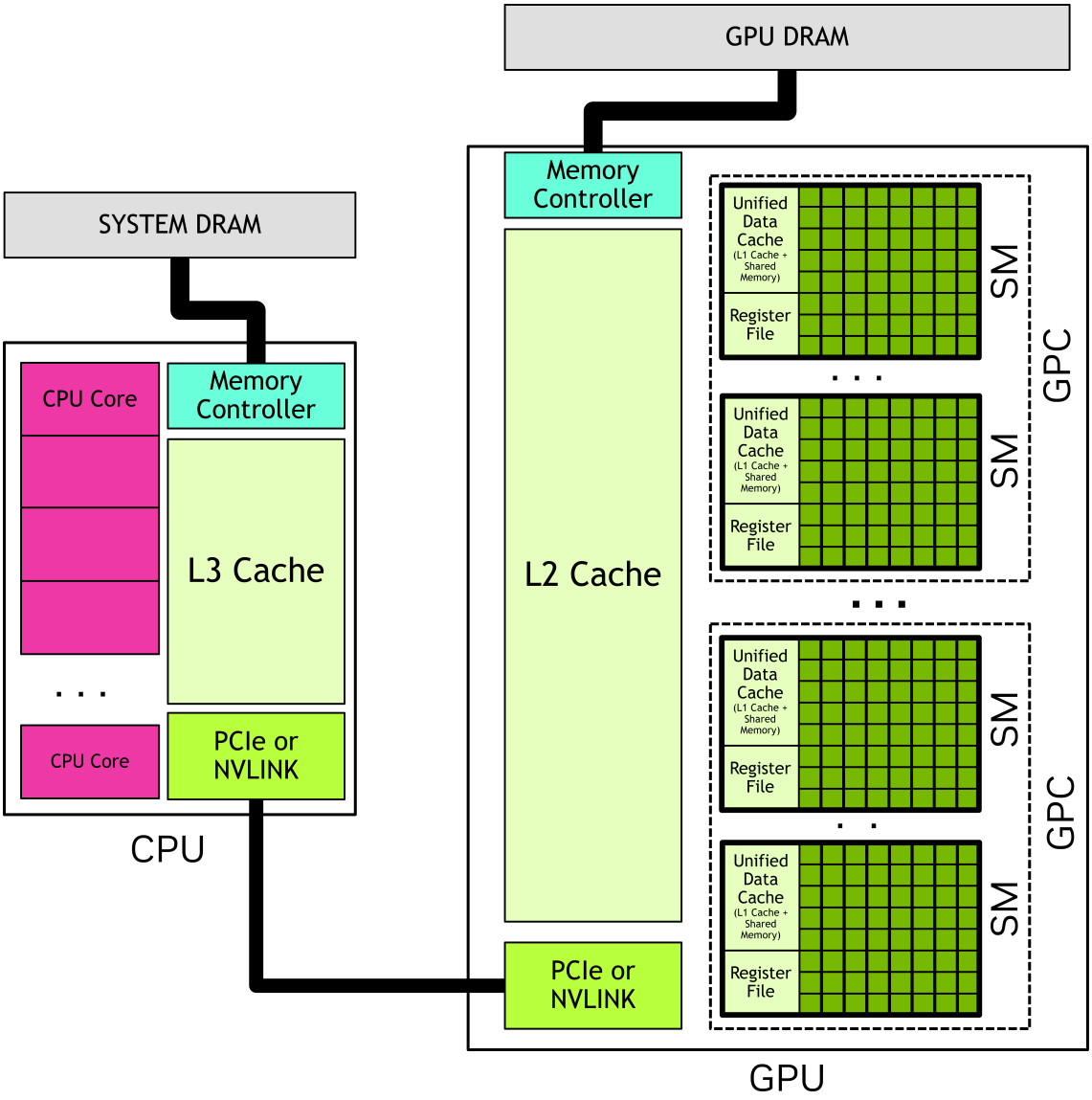


Figure 6.2: Overview of the GPU memory hierarchy and Streaming Multiprocessors (SMs) most relevant to this work [18].

6.3 Hardware constraints

As shown in Figure 6.2, the CPU and GPU are connected through PCIe, which introduces substantially higher latency and lower bandwidth than on-device memory accesses. Consequently, unnecessary host-device data transfers can significantly reduce performance and should be minimised. Although some datacentre systems additionally support NVLink interconnects, the hardware used in this work provides only PCIe connectivity, and therefore NVLink-specific optimisations are not considered further.

The experiments in this work were performed on an NVIDIA GPU based on the Blackwell 2.0 architecture (GB206). On this architecture, peak double-precision (FP64) throughput is reduced to 1/64th of peak single-precision (FP32) throughput [19]. This behaviour is typical of modern consumer GPUs, where hardware resources are primarily optimised for lower precision arithmetic. In contrast, datacentre-oriented Blackwell GPUs such as the NVIDIA GB200 provide FP64 throughput equal to one half of FP32 throughput [17].

This disparity motivates parameterising the generated device code on `ValueType`, allowing the numerical precision used by the solvers to be selected independently of the generated implementation.

6.4 CUDA Streams

CUDA streams provide a mechanism for organising work submitted to the GPU. A stream is an ordered sequence of operations, including kernel launches, memory transfers, and synchronisation events, which execute sequentially with respect to each other. Operations issued to different streams may execute independently, allowing separate computations to proceed without being blocked by synchronisation in other streams. This capability is used in the CVODE implementation, where multiple independent solves are assigned to different streams; the details and motivation for this design are discussed in Chapter 7.4.

Chapter 7

GPU Time-Stepping Solvers

7.1 Integration of Generated Device Code into the GPU Solver

The host launches the Forward Euler solve through `EulerKernelLauncher`, which computes the grid size from the number of cells (as described in Section 6.1) and launches `ComputeOdeSolution`. The block size is controlled by the static `mThreadsPerBlock` member, allowing the launch configuration to be adjusted at runtime for efficiency on different GPU architectures.

```
static int GetGridSize(int N)
{
    return (N + mThreadsPerBlock - 1) / mThreadsPerBlock;
}

static void LaunchComputeOdeSolution(...)
{
    ...
    BackwardEulerKernels::ComputeOdeSolution<CellType, ValueType, ...>
        <<<GetGridSize(N), mThreadsPerBlock>>>(...);
    ...
}
```

The launched kernel batches the solve across cells. Each thread loads the state of a single cell from the device array `ysPtr` into a thread-local array `y_local`, repeatedly applies `EvaluateYDerivativesDevice` to perform the Forward Euler update in Equation 2.5, and writes sampled outputs back to device memory. All data required by `EvaluateYDerivativesDevice` must therefore be passed explicitly as kernel arguments so that it can be forwarded during the timestep loop. This includes per-cell parameters, flags, and auxiliary data used within the update routines.

These arguments consist primarily of small scalar values passed by value and pointers to arrays stored in device memory. Several of these arrays represent two-dimensional cell data, such as model parameters or sampled outputs, where each cell stores multiple values. As discussed in Section 6.2, accesses to these arrays are most efficient when memory accesses are coalesced across threads. Consequently, the generated kernels are additionally templated on Boolean layout parameters controlling whether parameter and output arrays are interpreted using an Array-of-Structs (AoS) or Struct-of-Arrays (SoA) layout, illustrated in Figure 7.1.

This flexibility increases the complexity of the solver interface, resulting in the relatively large kernel signature shown below:

```
template<
    typename CellType,
    typename ValueType,
    bool ParametersSoA,
    bool OutputsSoA>
__global__
static void ComputeOdeSolution(

    // Time-stepping information
    const ValueType dt,
    const ValueType tStart,
    const int numCells,

    // Per-cell model parameters
    const ValueType* parameters_ptr,
    const unsigned nParameters,

    // Per-cell voltage clamp state
    const unsigned char* voltageDerivativeToZeroFlagsPtr,
    const ValueType* fixedVoltagePtr,

    // Current cell states on device
    ValueType* ys_ptr,

    // Shared simulation parameter
    const ValueType capacitance,

    // Output sampling configuration
    const unsigned n_steps,
    const unsigned n_small_steps,
    const ValueType tSamp,

    // Sampled outputs
    ValueType* out_solutions_ptr,
    ValueType* out_times_ptr,

    // Device-compatible stimulus evaluator
    const DeviceStimulusFunctor<ValueType> stimulus);
```

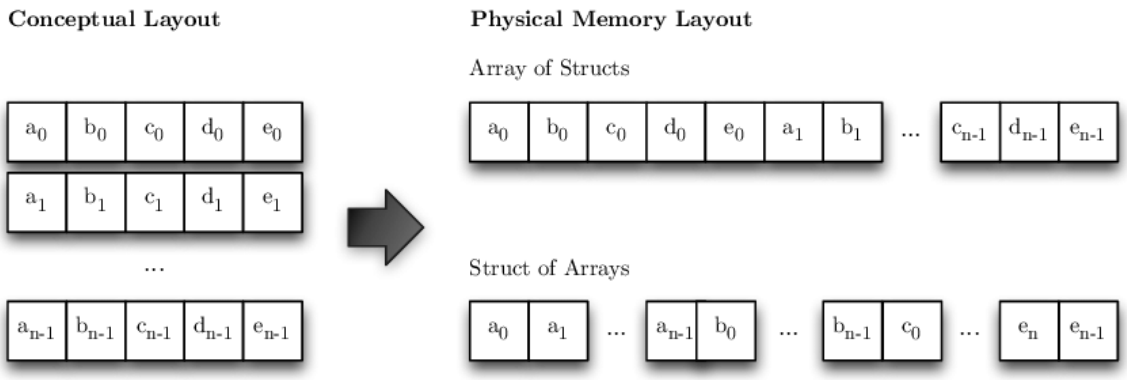


Figure 7.1: Comparison of Array-of-Structs (AoS) and Struct-of-Arrays (SoA) layouts for cardiac cell data. AoS stores values contiguously per cell, whereas SoA stores corresponding variables contiguously across all cells [21].

The optimal memory layout for overall efficiency depends on how each array is used during the solve. Arrays such as `voltageDerivativeToZeroFlagsPtr` and `fixedVoltagePtr` are one-dimensional in `numCells`, so neighbouring threads naturally access neighbouring elements. These accesses are therefore already coalesced without requiring any additional reorganisation.

In contrast, arrays such as `ys_ptr`, `parameters_ptr`, and `out_solutions_ptr` represent two-dimensional data: multiple values per cell across all cells. For these arrays, the choice between AoS and SoA layouts affects whether adjacent threads access adjacent memory locations. In the SoA layout, when all threads access the same variable, memory accesses are naturally coalesced. However, adopting this layout introduces an additional cost, since data must be reorganised either during host-device transfers or within the kernel itself.

The impact of this trade-off differs between arrays. The state array `ys_ptr` is read once at the start of the kernel, where each thread gathers the state of a single cell into the thread-local array `y_local`. After this point, all updates are performed using the local copy, and no further reads from `ys_ptr` are required. Consequently, the benefit of reorganising the state array is limited, since the improved access pattern is exploited only during a single read.

In contrast, model parameters are accessed repeatedly throughout the timestep loop. Each evaluation of the ionic currents and gating variables may involve multiple parameter reads, so improvements in memory access efficiency are applied many times. In this case, the cost of converting the layout can be amortised over the entire solve.

Finally, `out_solutions_ptr` represents the largest data structure in the solver, storing the full state of all cells at every output timestep. Although written less frequently than parameters are read, it dominates the overall memory traffic due to its size, and must also be transferred back to the host and converted into the AoS-like layout naturally used by the CPU-side cell objects. For smaller models such as Luo–Rudy 1991, this reconstruction can dominate the total runtime, whereas for larger models the computational cost of the solve becomes more significant. This trade-off motivates the performance experiments presented in Section 8.3.

The choice of layout is made at runtime by the host-side launcher, which dispatches to one of four compile-time kernel instantiations corresponding to the possible combinations of `ParametersSoA` and `OutputsSoA`. This avoids introducing additional branching within the kernel timestep loop while still allowing the memory layout to be configured experimentally.

The generated derivative functions were introduced earlier as taking a parameter array. In the implementation, this is generalised slightly by passing a lightweight accessor object rather than a raw pointer. The generated code continues to use ordinary subscript syntax, while the accessor controls how indices are mapped onto memory for either AoS or SoA layouts.

```
template<typename ValueType>
struct AoSArrayAccessor {
    __device__
    ValueType operator[](unsigned i) const {
        return data[index * stride + i];
    }
};

template<typename ValueType>
struct SoAArrayAccessor {
    __device__
    ValueType operator[](unsigned i) const {
        return data[i * stride + index];
    }
};
```

Both accessors expose the same `operator[]` interface but implement different memory mappings. Consequently, the generated derivative routines remain independent of the chosen parameter layout while continuing to use ordinary `parameters[i]` accesses.

Each kernel thread first determines which cell it is responsible for, allocates thread-local storage for that cell state, and gathers the initial state from the AoS-organised

device state array into thread-local memory:

```
int cell_idx = blockIdx.x * blockDim.x + threadIdx.x;
if (cell_idx >= numCells) return;

constexpr unsigned TOTAL_SIZE = CellType::TOTAL_SIZE;

ValueType y_local[TOTAL_SIZE];
ValueType dy_local[TOTAL_SIZE];

for (unsigned var_idx = 0; var_idx < TOTAL_SIZE; ++var_idx) {
    y_local[var_idx] = ys_ptr[(size_t)cell_idx * TOTAL_SIZE + var_idx];
}
```

The parameter accessor is then constructed. Although the accessor type is selected at compile time through `if constexpr`, `const auto` allows the result to be bound to a single local variable. The result is either an `AoSArrayAccessor` or a `SoAArrayAccessor`, both of which expose the same subscript interface to the generated device code. The per-cell fixed-voltage flag and value are also loaded here; these are passed through to the generated model code, where they can suppress the voltage derivative and use a fixed membrane potential when required.

```
const auto parameters = [&]() {
    if constexpr (ParametersSoA) {
        return SoAArrayAccessor<ValueType>(...);
    } else {
        return AoSArrayAccessor<ValueType>(...);
    } }();

const bool setVoltageDerivativeToZero =
    (voltageDerivativeToZeroFlagsPtr[cell_idx] != 0u);
const ValueType fixedVoltage = fixedVoltagePtr[cell_idx];
```

The main loop advances the cell through each sampled output interval. Within each interval, the generated `EvaluateYDerivativesDevice` routine is repeatedly evaluated and the resulting derivatives are used to update the thread-local state through the Forward Euler update. Sampled outputs are then written using either the SoA or AoS output layout. The `if constexpr` used when writing sampled outputs ensures that the layout branch is resolved at compile time rather than runtime, so separate specialised kernels are generated for each layout combination and the unused branch is eliminated entirely by the compiler.

```
// Loop over output samples. Each sample is separated by tSamp.
for (unsigned sample = 0; sample < n_steps; ++sample) {
    ValueType curr_time = tStart + static_cast<ValueType>(sample) * tSamp;
    // Take multiple sub steps between each sampled point in time
    // Here, n_small_steps = tSamp / dt
    for (unsigned sub = 0; sub < n_small_steps; ++sub) {
        curr_time = tStart + static_cast<ValueType>(sample) * tSamp
            + static_cast<ValueType>(sub) * dt;

        // Template argument deduction infers ValueType and ParameterAccessor
        // directly from the function arguments
        CellType::EvaluateYDerivativesDevice(
            curr_time,
            y_local,  // y_n
            dy_local, // Location to write f(y_n, t_n) to
            setVoltageDerivativeToZero,
            fixedVoltage,
            parameters,
            capacitance,
            stimulus);
        // y_{n+1} = y_n + dt * f(y_n, t_n)
        for (unsigned var_idx = 0; var_idx < CellType::TOTAL_SIZE; ++var_idx) {
            y_local[var_idx] += dt * dy_local[var_idx];
        }
    }
    if (cell_idx == 0) { out_times_ptr[sample] = curr_time + dt; }

    size_t sample_base = static_cast<size_t>(sample) * static_cast<size_t>(
        TOTAL_SIZE) * static_cast<size_t>(numCells);

    for (unsigned var_idx = 0; var_idx < TOTAL_SIZE; ++var_idx) {
        if constexpr (OutputsSoA) {
            out_solutions_ptr[sample_base + static_cast<size_t>(var_idx)
                * static_cast<size_t>(numCells) + static_cast<size_t>(cell_idx)]
                = y_local[var_idx];
        } else {
            out_solutions_ptr[sample_base + static_cast<size_t>(cell_idx)
                * static_cast<size_t>(TOTAL_SIZE) + static_cast<size_t>(var_idx)]
                = y_local[var_idx];
        }
    }
}
```

Hence, `ComputeOdeSolution` repeatedly evaluates the generated device derivative routine to advance the system and write sampled solutions back to device memory.

7.2 Host-Side Integration and Data Management

The remaining challenge is integrating these GPU solver kernels into the existing CPU-oriented Chaste workflow. This requires gathering state and parameter data from CPU-side cell objects, transferring this data to the GPU, launching the appropriate solver kernels, and reconstructing `OdeSolution` objects from the sampled device outputs.

The `CardiacCellCollection` abstraction responsible for this integration was introduced earlier in Chapter 3; the focus here is therefore on the implementation details: host-device memory management, kernel dispatch, and solution reconstruction.

```
template<typename CellType, typename ValueType>
class CardiacCellCollection : public AbstractCardiacCellCollection<ValueType>
{
public:
    // Method-specific GPU launcher supplied by the generated cell type
    using SolverKernelLauncher =
        typename CellType::template SolverKernelLauncher<ValueType>;

    using Base = AbstractCardiacCellCollection<ValueType>;
    using typename Base::Timings; // Benchmark timing statistics
    using typename Base::ComputeParams; // Timestep, sampling, and layout options

    // Compile-time state-vector size supplied by the generated cell type
    static constexpr unsigned TOTAL_SIZE = CellType::TOTAL_SIZE;

    CardiacCellCollection(std::vector<std::unique_ptr<CellType>>&& cells);

    // Main entry point: run the GPU solve and return one solution per cell
    std::vector<OdeSolution> Compute(double tStart, double tEnd,
        const ComputeParams& params = ComputeParams()) override;

private:
    // Owned CPU-side cell objects used for configuration and reconstruction
    std::vector<std::unique_ptr<CellType>> mCells;
};
```

The `CardiacCellCollection` constructor takes the cell vector by rvalue reference (`&&`), transferring ownership of the heap-allocated cell objects into the collection without copying.

```
template<typename CellType, typename ValueType>
CardiacCellCollection<CellType, ValueType>::CardiacCellCollection(
    std::vector<std::unique_ptr<CellType>>&& cells)
{
    for (std::unique_ptr<CellType>& cell : cells) { AddCell(std::move(cell)); }
}
```

Rather than constructing separate GPU-specific cell representations, the collection stores ordinary Chaste cardiac cell objects and reuses their existing interfaces for parameter configuration, stimulus configuration, and state management. This allows existing CPU-oriented setup code to be reused directly when constructing GPU simulations.

Additional solver configuration is collected into a small parameter structure shared by the GPU collection classes:

```
struct ComputeParams
{
    Timings* timings;

    double dt;            // Internal ODE timestep
    double tSamp;         // Output sampling interval

    bool parametersSoA;
    bool outputsSoA;

    // Used only by CvodeCardiacCellCollection
    unsigned batchSize;
    bool useSparse;
};
```

Both `CardiacCellCollection` and `CvodeCardiacCellCollection` derive from the shared `AbstractCardiacCellCollection<ValueType>` hierarchy, which provides common configuration structures such as `ComputeParams`.

The concrete `CardiacCellCollection<CellType,ValueType>` manages the host-device memory transfers, kernel launches, and solution reconstruction required to simulate generated GPU-enabled cardiac cell models. Runtime-sized device buffers are stored using `thrust::device_vector`, with `thrust::copy` used for host-device transfers [4]. The overall structure of `Compute` is shown below.

```
template<typename CellType, typename ValueType>
std::vector<OdeSolution>
CardiacCellCollection<CellType, ValueType>::Compute(
    double tStart,
    double tEnd,
    const ComputeParams& params)
{
    // 1. Validate configuration and compute timestep structure

    std::vector<OdeSolution> all_solutions;
    if (mCells.empty()) return all_solutions;
    const unsigned num_cells = Size();

    const StepConfig step_config =
    ComputeStepConfig(tStart, tEnd, params.dt, params.tSamp);
```

```
// Validated timestep and sampling configuration
const double dt = step_config.dt;
const double tSamp = step_config.tSamp;
const unsigned n_steps = step_config.nSteps;          // (tEnd-tStart)/tSamp
const unsigned n_small_steps = step_config.nSmallSteps; // tSamp/dt

// 2. Gather CPU-side state and transfer to device memory

// Gather CPU-side cell states into an AoS host buffer
std::vector<ValueType> h_ys_aos;
GatherCellStates(h_ys_aos);
thrust::device_vector<ValueType> d_ys = h_ys_aos;

// 3. Pack per-cell parameters and initialise shared parameters

// Retrieve membrane capacitance from the current HeartConfig
InitCapacitance();

thrust::device_vector<ValueType> d_params;
thrust::device_vector<unsigned char> d_voltage_derivative_to_zero_flags;
thrust::device_vector<ValueType> d_fixed_voltage;

unsigned n_parameters = 0;
PackPerCellParametersToDevice(...);

// 4. Allocate sampled output buffers on device

thrust::device_vector<ValueType> d_out_solutions(
    static_cast<size_t>(n_steps) * TOTAL_SIZE * num_cells);
thrust::device_vector<ValueType> d_out_times(n_steps);

// 5. Extract raw device pointers for CUDA kernel launch

ValueType* ys_ptr = thrust::raw_pointer_cast(d_ys.data());
const ValueType* params_ptr = thrust::raw_pointer_cast(d_params.data());
const unsigned char* voltage_derivative_flags_ptr =
    thrust::raw_pointer_cast(d_voltage_derivative_to_zero_flags.data());
const ValueType* fixed_voltage_ptr =
    thrust::raw_pointer_cast(d_fixed_voltage.data());
ValueType* out_solutions_ptr = thrust::raw_pointer_cast(
    d_out_solutions.data());
ValueType* out_times_ptr = thrust::raw_pointer_cast(d_out_times.data());

// 6. Construct device-compatible stimulus representation

std::vector<StimulusDescriptor> stimulus_descs =
    mCells[0]->GetIntracellularStimulusDescriptor();
DeviceStimulusFunctor<ValueType> device_stimulus(stimulus_descs);

// 7. Launch the GPU solver kernel

SolverKernelLauncher::LaunchComputeOdeSolution(...);

// 8. Copy sampled outputs back to the host
```

```
    std::vector<ValueType> h_times(n_steps);
    thrust::copy(d_out_times.begin(), d_out_times.end(), h_times.begin());
    std::vector<ValueType> h_out(
        static_cast<size_t>(n_steps) * TOTAL_SIZE * num_cells);
    thrust::copy(d_out_solutions.begin(), d_out_solutions.end(), h_out.begin());

    // 9. Reconstruct OdeSolution objects and update CPU-side state

    ReconstructSolutions(h_times, h_out, all_solutions, params.outputsSoA);
    return all_solutions;
}
```

The collection performs a single host-device transfer of the initial state and parameter data, a single GPU kernel launch, and a single device-host transfer of the sampled outputs, minimising both PCIe transfers and kernel launches as discussed in Section 6.3.

The dominant device allocation is typically `d_out_solutions`, whose size scales with the number of cells, sampled timesteps, and model state variables:

$$\texttt{sizeof(ValueType)} * \texttt{n_steps} * \texttt{TOTAL_SIZE} * \texttt{num_cells}.$$

Sampled outputs are transferred back to the host in their device-side layout before reconstruction into `OdeSolution` objects. As shown later in Section 8.3, this reconstruction can become comparable in cost to the numerical solve for smaller models.

7.3 Backward Euler solver

The Backward Euler implementation reuses the same overall GPU execution model introduced for the Forward Euler solver. The host-side `CardiacCellCollection`, memory layouts, parameter accessors, and kernel launch structure remain unchanged. Only the internal timestep update performed within `ComputeOdeSolution` differs.

As discussed earlier in Section 2.3, each timestep first updates the membrane voltage explicitly, then advances the remaining state variables using the semi-implicit Backward Euler formulation generated in Section 4.3. Consequently, compared with the Forward Euler kernel, the Backward Euler implementation requires additional thread-local storage for the nonlinear subsystem, including the Newton iterate, residual, Jacobian, and update vectors:

```
ValueType y_local[CellType::TOTAL_SIZE];

// nonlinear_size = compile-time max(1, CellType::NONLINEAR_SIZE)
constexpr unsigned nonlinear_size = ...;
ValueType guess[nonlinear_size];
ValueType residual[nonlinear_size];
ValueType update[nonlinear_size];
ValueType jacobian[nonlinear_size][nonlinear_size];

for (unsigned sub = 0; sub < nSmallSteps; ++sub) {

    CellType::UpdateTransmembranePotentialDevice(...);

    CellType::SolveClosedFormVarsDevice(...);

    if constexpr (CellType::NONLINEAR_SIZE > 0) {

        CellType::FillInitialGuessDevice(...);

        CardiacNewtonSolverDevice<CellType, ValueType>::Solve(...);

        CellType::ScatterSolutionDevice(...);

    }
}
```

The Newton solver itself closely mirrors the existing CPU implementation in Chaste, using the generated residual and Jacobian routines to solve the reduced system in Equation 2.19. A device-compatible implementation is included in Appendix B.

Hence, with only minimal changes to the kernel timestep update and thread-local state allocations, the same overall GPU execution model can also be used to simulate generated CellML models using the modified Backward Euler method. More generally, this structure makes it straightforward to extend the framework to additional numerical integration methods in the future.

7.4 GPU-Enabled CVODE Integration

Sections 2.5 and 4.4 introduced the mathematical structure of the batched CVODE system together with the generated device-compatible derivative and Jacobian routines required for GPU execution. The remaining task is therefore to integrate these generated kernels into the GPU-enabled SUNDIALS interface and construct the corresponding batched solve workflow within Chaste.

Unlike the fixed-step Forward Euler and Backward Euler solvers introduced previously, CVODE internally manages timestep selection, Newton iteration, and linear solver execution on the host. Consequently, the GPU implementation does not consist of a single long-running solver kernel. Instead, CVODE repeatedly launches user-provided GPU routines implementing the combined right-hand side system $\mathbf{F}(\mathbf{Y})$ from Equation 4.1 and the block-diagonal Jacobian J from Equation 4.2 throughout the adaptive BDF solve.

The implementation therefore introduces a new `BatchedCvodeSystem<CellType, ValueType>` abstraction, responsible for exposing these GPU evaluation routines through the SUNDIALS callback interface. Internally, the class stores references to the batched device-side state, parameter arrays, Jacobian storage, CUDA streams, and the GPU-enabled SUNDIALS linear solver objects.

Two GPU kernels are provided to SUNDIALS: `BatchedRhsKernel`, which evaluates the combined right-hand side system, and `BatchedJacKernel`, which constructs the block-diagonal Jacobian. The right-hand side kernel follows the same structure as the Forward Euler implementation, with each GPU thread evaluating `EvaluateYDerivativesDevice` for a single cell in the batch. The Jacobian kernel is slightly more informative and is shown below. Each thread constructs either a dense or sparse Jacobian block for one cell before writing it into the larger block-diagonal matrix expected by the GPU-enabled SUNDIALS linear solver interface.

```
template<typename CellType, typename ValueType, bool UseSparse>
__global__
void BatchedJacKernel(
    unsigned batchSize,
    unsigned nVarsPerCell,
    const ValueType* yData,
    ValueType* blockJacobianData,
    realtype time,
    ...)
{
    unsigned cell_idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (cell_idx >= batchSize) return;

    const ValueType* y_cell = yData
        + static_cast<size_t>(cell_idx) * nVarsPerCell;
    ValueType* jacobian_block;

    if constexpr (UseSparse) {

        jacobian_block = blockJacobianData
            + static_cast<size_t>(cell_idx) * CellType::NNZ;

    } else {
```

```
        jacobian_block = blockJacobianData
            + static_cast<size_t>(cell_idx) * nVarsPerCell * nVarsPerCell;
    }

    AoSArrayAccessor<ValueType> parameters(parametersPtr, nParametersPerCell,
        cell_idx);

    if constexpr (UseSparse) {

        CellType::EvaluateSparseAnalyticJacobianDevice(...);

    } else {

        auto jacobian_dense = reinterpret_cast<
                ValueType (*)[CellType::TOTAL_SIZE]>(jacobian_block);

        CellType::EvaluateAnalyticJacobianDevice(...);
    }
}
```

The corresponding `CvodeCardiacCellCollection::Compute` method partitions the collection into independent batches and launches a separate `BatchedCvodeSystem` for each batch.

The full `BatchedCvodeSystem` implementation additionally contains substantial setup code required by the experimental GPU-enabled SUNDIALS interface, including CUDA-aware `N_Vectors` (SUNDIALS' vector wrapper), cuSPARSE block matrices, cuSOLVER linear solvers, and CUDA execution policies. The complete implementation is included in Appendix C. The overall execution pattern, however, remains relatively simple: independent CVODE batches are launched concurrently using separate CPU threads and CUDA streams, as shown in Listing 14.

This allows multiple adaptive BDF solves to progress concurrently whilst overlapping GPU kernel execution across streams. Unlike the fixed-step solvers described previously, however, CVODE periodically transfers the batched state back to the host after accepted output timesteps so that `OdeSolution` objects may be reconstructed incrementally. Consequently, the implementation performs substantially more host-device synchronisation and memory movement than the Forward Euler and Backward Euler collections, which becomes an important factor in overall performance.

```
for (unsigned batch_start = 0; batch_start < num_cells; batch_start += batch_size)
{
    futures.push_back(std::async(
        std::launch::async, [=, &all_solutions, &h_ys]()
    {
        cudaStream_t local_stream;

        // Each batch uses its own non-blocking CUDA stream.
        cudaStreamCreateWithFlags(&local_stream, cudaStreamNonBlocking);

        auto* local_policy =
            new SUNCudaThreadDirectExecPolicy(
                128,             // CUDA block size
                local_stream);   // Use our own instruction stream

        BatchedCvodeSystem<CellType, ValueType> batched(
            this_batch,          // Number of cells in this batch
            TOTAL_SIZE,          // State variables per cell
            p_batch_volt_flags,  // Per-cell fixed-voltage flags
            p_batch_fixed_volt,  // Per-cell fixed-voltage values
            p_batch_params,      // Per-cell model parameters
            nparams,             // Parameters per cell
            this->mCapacitance,  // Shared capacitance value
            h_stim,              // Device-compatible stimulus functor
            local_stream,        // Stream used by callback kernels
            local_policy,        // SUNDIALS CUDA execution policy
            use_sparse);         // Dense or sparse Jacobian path

        N_Vector y = N_VNew_Cuda(...);

        // SUNDIALS distinguishes between the policy used for ordinary vector
        // kernels and the policy used for reductions; both use the same stream.
        N_VSetKernelExecPolicy_Cuda(y, local_policy, local_policy);

        N_VCopyToDevice_Cuda(y);

        batched.SetupCvode(
            y,
            static_cast<realtype>(tStart),
            static_cast<realtype>(batch_max_dt));

        batched.Solve(
            static_cast<realtype>(tStart),
            static_cast<realtype>(tEnd),
            static_cast<realtype>(batch_max_dt),
            static_cast<realtype>(tSamp),
            all_solutions,
            batch_start);
    }));
}
```

Listing 14: Simplified concurrent batched CVODE execution. Each worker thread owns an independent CUDA stream and `BatchedCvodeSystem` instance.

Chapter 8

Results and Discussion

8.1 GPU Infrastructure Validation

To validate the generated stimulus infrastructure and end-to-end GPU execution pipeline, we compared automatically generated GPU implementations against the existing validated CPU implementations within Chaste for the Luo–Rudy 1991 model, using both Forward Euler (FE) and Backward Euler (BE). The stimulus configurations are summarised in Table 8.1. Figures 4.2 and 4.3 respectively show the waveform shapes and GPU-generated action potentials for the first cell in Configurations 3–5.

Cfg.	Stimulus configuration	Parameters
1	Zero stimulus	No stimulus applied
2	Simple stimulus	$I = -25.5$, duration $= 2.0$ ms start time $= 100.0$ ms
3	Regular stimulus	$I = -25.5$, duration $= 2.0$ ms period $= 1000.0$ ms, start time $= 100.0$ ms
4	Zero-net-charge stimulus	$I = -50.0$, duration $= 4.0$ ms period $= 1000.0$ ms, start time $= 100.0$ ms
5	Multi-stimulus configuration	Regular: $I = -25.5$, duration $= 2.0$ ms, period $= 1000.0$ ms, start $= 100.0$ ms, stop $= 1102.0$ ms zero-net-charge: $I = -50.0$ duration $= 4.0$ ms, period $= 1000.0$ ms, start $= 2100.0$ ms, stop $= 3104.0$ ms

Table 8.1: Stimulus configurations used to evaluate the generated GPU stimulus infrastructure.

Simulations were performed for 4100 ms on batches of 32 cells, with the three model parameters randomly scaled by multipliers in $[0.8, 1.2]$ to exercise a wider range of numerical states. All configurations were evaluated in double precision using timesteps of 10^{-5} ms and 10^{-4} ms for FE and BE respectively. GPU results were compared against the corresponding automatically generated CPU implementations. Across all stimulus configurations and state variables, the generated GPU implementations closely matched the corresponding CPU implementations. Maximum absolute voltage errors remained below 1.1×10^{-10} mV, while mean absolute and RMS errors remained on the order of 10^{-15} mV and 10^{-13} mV respectively.

This indicates agreement up to negligible floating-point roundoff differences, supporting the correctness of the automatically generated model-specific device functions, GPU solver implementations, and supporting execution infrastructure.

8.2 Numerical Stability and Timestep Analysis

We next evaluate the numerical behaviour of the generated CUDA FE and BE solvers using the representative cardiac cell models shown in Table 8.2.

Model	**Year**	**Total states**	**Nonlinear size**
Hodgkin–Huxley (H–H) [12]	1952	4	0
Luo–Rudy (L–R) [16]	1991	8	1
Faber–Rudy (F–R) [8]	2000	25	11
ToR–ORd Endo (T–O) [24]	2019	45	19

Table 8.2: Representative cardiac cell models used to evaluate the solvers. Nonlinear size corresponds to the number of variables remaining in the BE Newton solve.

Using the same 32-cell parameterised batches described previously, timestep sweeps were performed for each model using a regular stimulus producing two pacing events over a 2 s simulation.

Before presenting the timestep sweeps, Figure 8.1 shows a representative GPU-generated ToR–ORd action potential obtained using the smallest tested BE timestep. The corresponding FE trace is visually indistinguishable. Despite the differing quantitative errors observed later, both solutions preserve physiological action-potential behaviour.

For the numerical comparisons, the pre-existing CPU implementation was evaluated in double precision, while the generated CUDA implementation was evaluated in

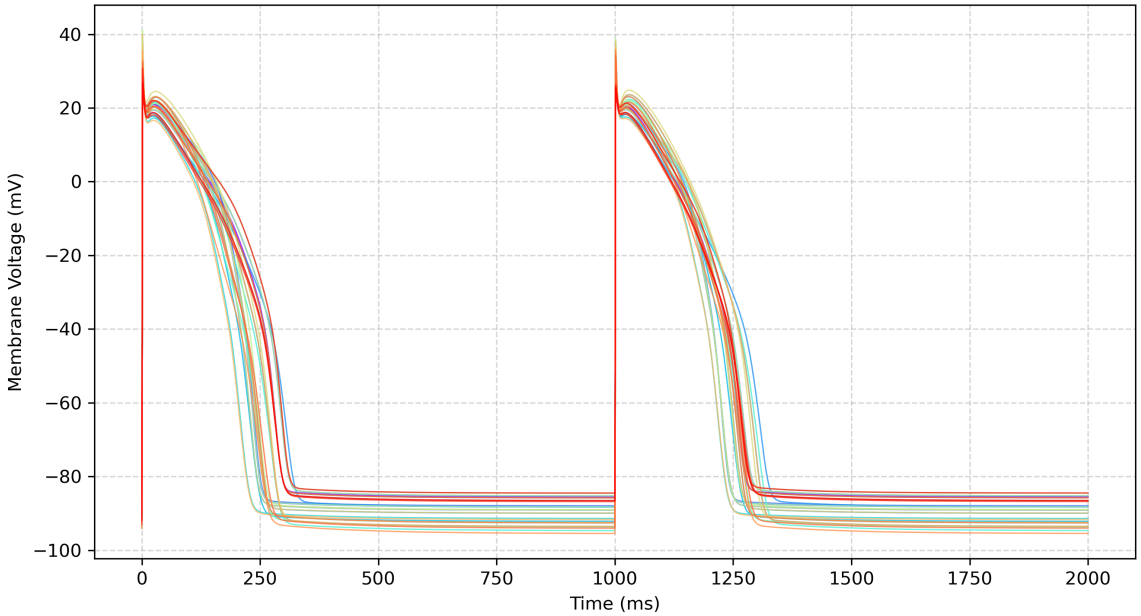


Figure 8.1: Representative GPU-generated ToR–ORd action potential using the BE solver with $\Delta t = 10^{-3}$ ms.

both single and double precision. Errors were measured relative to reference solutions computed using the CPU implementation with timesteps of 10^{-5} ms and 10^{-3} ms for FE and BE respectively. Figures 8.2 and 8.3 show the maximum absolute voltage error together with the maximum absolute error across all remaining state variables over the full batch and simulation duration.

The reference timestep is omitted from the CPU curves since its error is zero by construction. Errors are additionally capped at 10^2; values reaching this threshold indicate either errors exceeding 10^2 or numerical failure, including NaNs, infinities, invalid probabilities outside $[0, 1]$, or intrinsically positive state variables becoming negative.

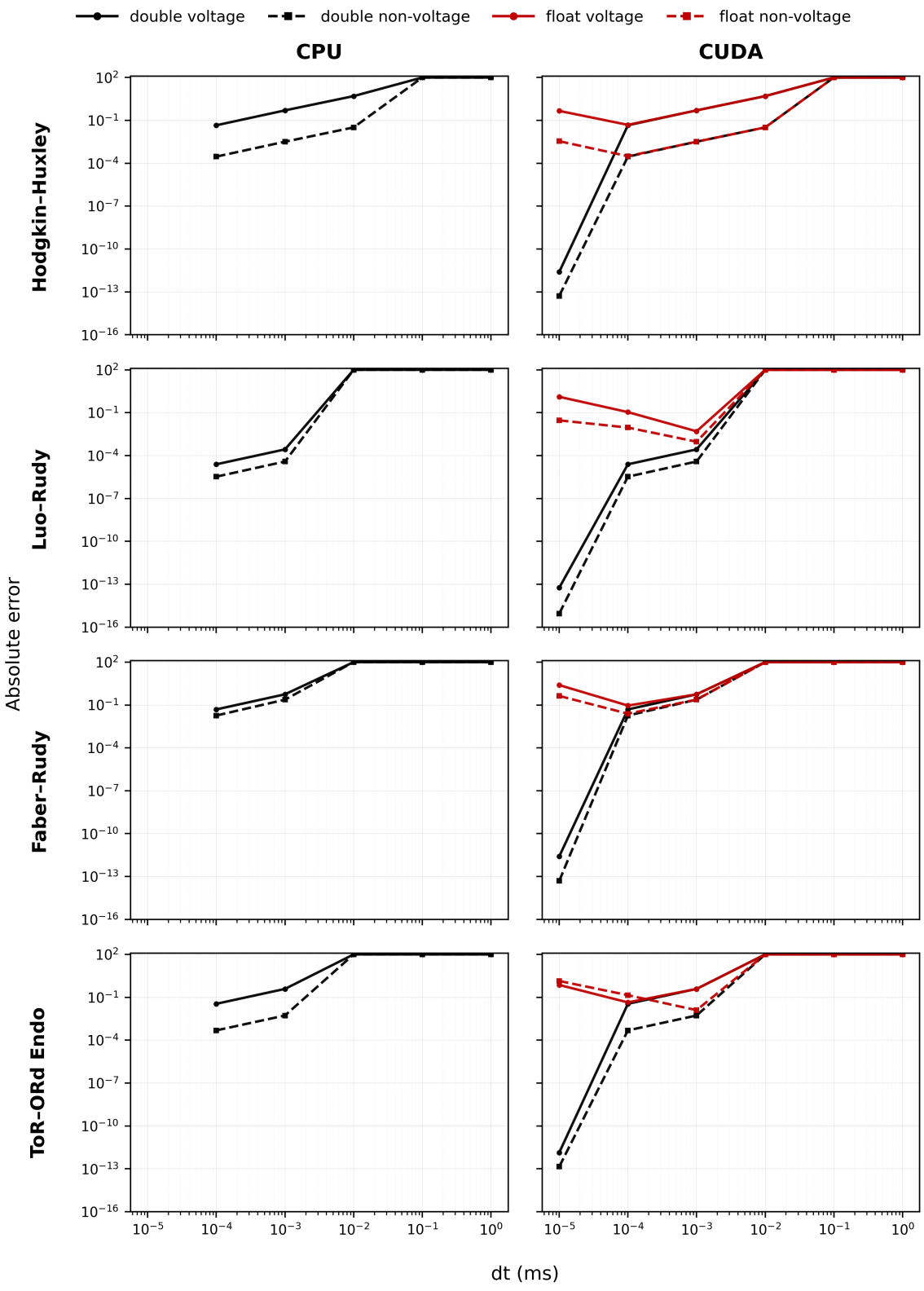


Figure 8.2: FE timestep sweeps across the representative cardiac cell models.

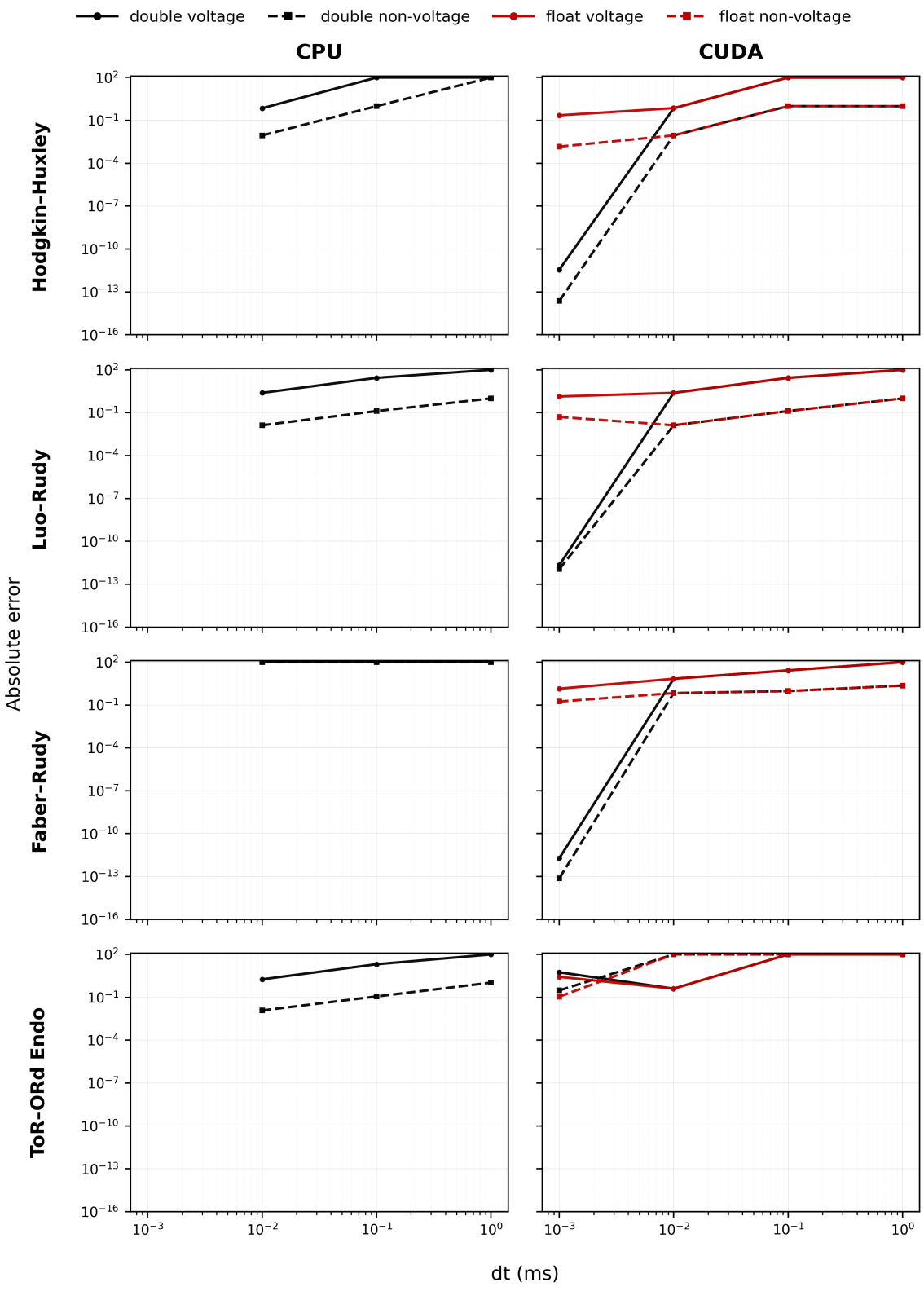


Figure 8.3: BE timestep sweeps across the representative cardiac cell models.

The generated CUDA FE and BE implementations closely reproduce the numerical behaviour and stability limits of the existing CPU implementations across all four models, with minor double-precision discrepancies remaining as expected due to hardware-specific instruction sequences. Across all configurations, absolute errors are dominated by the transmembrane voltage variable. For FE (Figure 8.2), both implementations remain stable across a broad timestep range. However, single-precision results degrade at the smallest timesteps ($\Delta t < 10^{-4}$ ms) due to accumulating rounding errors, often optimising at intermediate timesteps. This demonstrates that single precision remains practical for explicit integration, provided a sufficiently large timestep is chosen to avoid excessive rounding error accumulation.

The BE sweeps (Figure 8.3) exhibit the expected improvement in stability, achieving comparable accuracy to the finest FE sweeps while using substantially larger timesteps. Due to the high computational cost of the implicit solver, these sweeps were bounded at $\Delta t = 10^{-3}$ ms. While the simpler models track the reference closely at this bound, the more complex ToR–ORd model reaches a maximum voltage error of 5.616 mV. However, the corresponding action potential in Figure 8.1 remains visually indistinguishable from the reference; this suggests that much of the observed error arises from a minor temporal shift, such as a delayed upstroke, meaning the solution remains physiologically viable rather than exhibiting catastrophic numerical failure. Crucially, precision viability proves highly solver-dependent: while single precision is adequate for FE, single-precision BE configurations diverge rapidly from the double-precision solutions as model complexity increases, demonstrating that single precision is substantially less suitable for the implicit solver implementation used here.

8.3 GPU Performance Evaluation

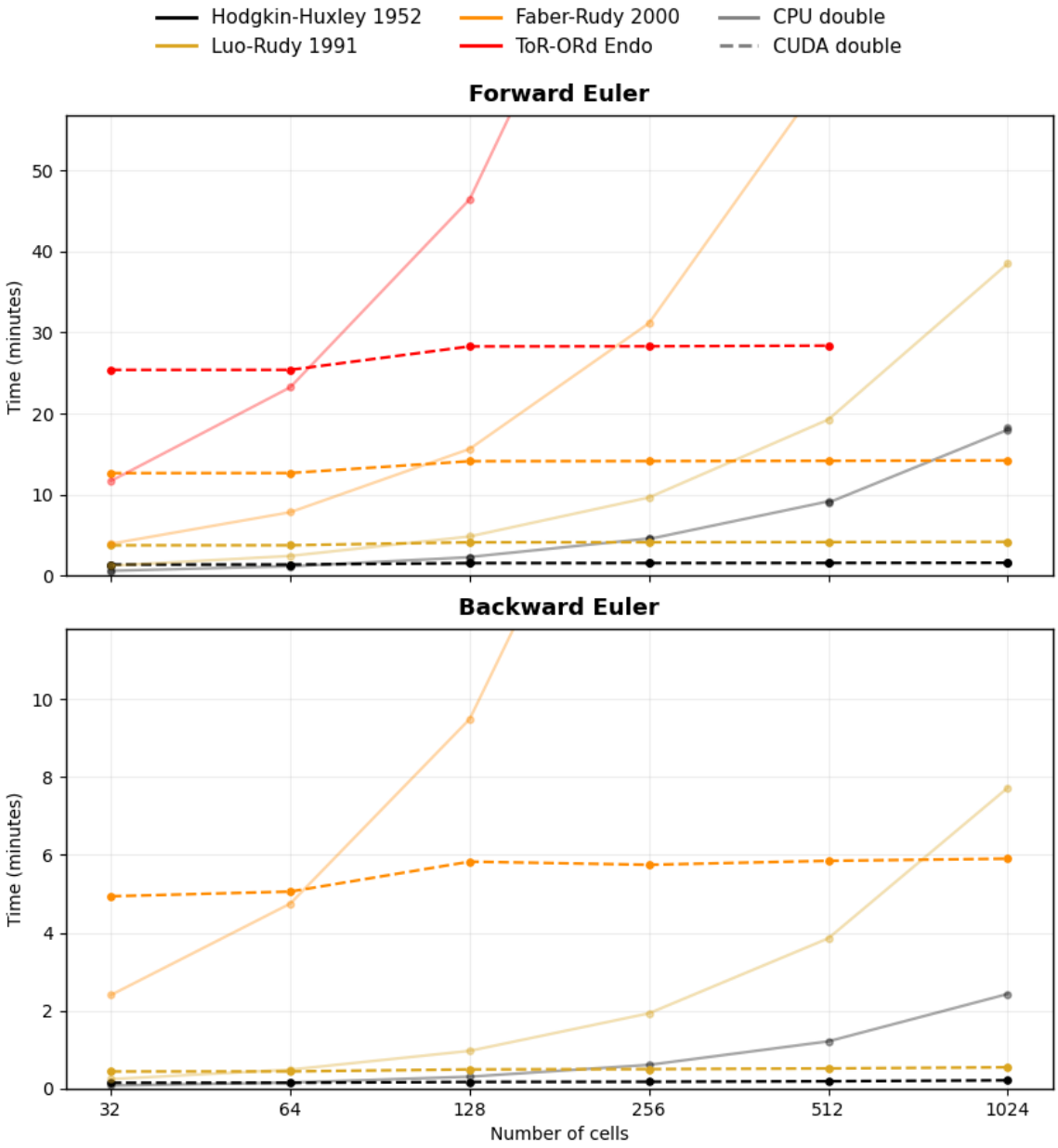
We now evaluate the performance of the generated CUDA implementations relative to the existing CPU implementations. Benchmarks were performed using the cardiac cell models from Table 8.2 with structure-of-arrays (SoA) layouts for parameter and output storage, as described in Section 7.1. Testing was conducted on a laptop equipped with an Intel Core i7-13650HX CPU and an NVIDIA RTX 5070 Laptop GPU. GPU timings include the complete execution pipeline, including host–device transfers, kernel execution, and reconstruction of CPU-side state arrays.

To mitigate the effects of CUDA runtime initialisation, JIT kernel compilation, and transient caching, an un-timed warm-up execution was performed for each configu-

ration. Informed by the accuracy–performance trade-offs identified in the previous error sweeps, timesteps were fixed at $\Delta t = 10^{-4}$ ms for FE and $\Delta t = 10^{-3}$ ms for BE. Figure 8.4 (top) shows the resulting CPU and GPU runtimes across increasing batch sizes.

Across all tested configurations, the generated CUDA implementations provide substantial acceleration relative to the corresponding CPU implementations. The GPU runtimes increase comparatively slowly as the batch size grows, indicating that the GPU remains far from fully saturated over much of the tested range. Due to severe runtime constraints on the CPU, the largest ToR–ORd FE configuration was only evaluated up to 512 cells. For the simpler models, the BE benchmarks additionally achieve lower total runtimes than the corresponding FE configurations despite the greater cost per timestep, since the larger stable timestep substantially reduces the total number of integration steps required. ToR–ORd forms the main exception to this trend due to the significantly larger nonlinear subsystem involved in the implicit solve; consequently, ToR–ORd BE was omitted from the main scaling benchmark suite due to its exceptional computational cost.

As highlighted at maximum batch sizes in Figure 8.4 (bottom), precision configuration profoundly influences solver performance. Transitioning from double precision (FP64) to single precision (FP32) yields substantial speedups across all configurations. This effect is most prominent in the Luo–Rudy FE solver, which achieves a massive 19.8× acceleration. This severe penalty under FP64—and the subsequent massive acceleration under FP32—is directly explained by the hardware constraints of the laptop-grade RTX 5070 GPU, which features a heavily restricted double-precision throughput compared to its theoretical single-precision capacity, as discussed in Section 6.3. In contrast, the larger Faber–Rudy and ToR–ORd models see less pronounced speedups. Because these complex systems manage significantly more state variables and equations, the increased resource pressure may make them more susceptible to constraints such as register spilling or local memory latency, potentially shifting their execution profiles and preventing them from fully leveraging the GPU's expanded single-precision arithmetic capacity.



Model	Method	Cells	CPU (s)	FP64 (s)	$\frac{\text{CPU}}{\text{FP64}}$	FP32 (s)	$\frac{\text{FP64}}{\text{FP32}}$
H–H	FE	1024	1079.5	94.0	11.5×	23.3	4.0×
L–R	FE	1024	2309.9	248.0	9.3×	12.6	19.8×
F–R	FE	1024	7473.6	852.0	8.8×	324.1	2.6×
T–O	FE	512	11143.4	1702.0	6.5×	628.4	2.7×
H–H	BE	1024	145.6	12.6	11.5×	3.8	3.3×
L–R	BE	1024	463.5	32.7	14.2×	5.8	5.6×
F–R	BE	1024	4624.5	354.1	13.1×	46.9	7.6×

Figure 8.4: CPU and GPU runtime scaling across increasing batch sizes (top), with detailed performance and speedup comparisons highlighted at maximum batch sizes (bottom).

A smaller supplementary layout experiment evaluating structure-of-arrays (SoA) versus array-of-structures (AoS) formats for the Luo–Rudy implementation at 128 cells provided further context. The SoA layouts yielded minor performance gains, showing a maximum runtime reduction of approximately 10% for the single-precision solver and staying closer to 2% in all other configurations (data not shown). Timing decompositions indicated that kernel execution heavily dominates the overall cost for larger models and double-precision implementations. This suggests that host-side state or memory array reconstructions have a limited practical impact on performance for these configurations, indicating that optimization efforts remain best directed toward mathematical kernel execution.

8.3.1 CVODE validation and performance

The generated CUDA CVODE implementation was similarly validated against its CPU counterpart by simulating 32 cells with parameters randomly scaled in the range $[0.8, 1.2]$ for 1 s. At relative and absolute tolerances of 10^{-5} and 10^{-7}, the Luo–Rudy 1991 model yielded a maximum absolute voltage error of 1.48×10^{-3} mV (mean $< 3 \times 10^{-4}$ mV). The complex ToR–ORd model produced maximum voltage and non-voltage errors of approximately 0.2 and 0.5, respectively (mean voltage error $\approx 10^{-3}$ mV). Dense and sparse GPU configurations yielded matching error profiles. Because the adaptive timestep controller operates globally across the batch (Section 2.5), individual cell variations smooth out as system size increases, suggesting larger batches may require tighter tolerances to maintain reference agreement.

Performance evaluations, however, revealed a profound disparity: for a workload of 512 ToR–ORd cells (16 batches of 32 cells using the dense representation), the CPU implementation finished in approximately 34 seconds, whereas the unoptimised CUDA variant took over 3.3 hours. Without a full execution profile, we hypothesise this stems from the shared timestep constraint (Section 2.5) forcing an excessively high total step count, severely compounded by frequent host–device transfers, global synchronisations, and repetitive kernel launch overheads absent from our simpler FE and BE implementatins.

To investigate this batching constraint, a series of shorter experiments were conducted by simulating the first 20 ms of the ToR–ORd model to capture the stiff upstroke dynamics. As detailed in Table 8.3, grouping cells by similar electrophysiological features (sorting by G_{Na}, P_{Ca}, or G_{Kr} prior to batching) consistently reduced execution

times compared to a random ordering, mitigating the timestep restrictions imposed by the most demanding cell in a batch. Transitioning to a sparse matrix representation or a single batch of 512 cells yielded further runtime variations. Notably, the dense 512-cell single-batch configuration failed the required tolerance checks, demonstrating that the choice between dense and sparse representations can significantly influence numerical stability as the batch size increases. Consequently, because this baseline GPU implementation remains uncompetitive with both the highly optimised CPU library and our FE solver (which completed a full 512-cell simulation in roughly 28 minutes), a comprehensive benchmarking suite for CUDA CVODE was omitted.

Batch configuration	Ordering	Matrix type	Runtime (s)
16 batches of 32	Random	Dense	562
16 batches of 32	G_{Na}	Dense	528
16 batches of 32	P_{Ca}	Dense	509
16 batches of 32	G_{Kr}	Dense	439
16 batches of 32	Random	Sparse	263
1 batch of 512	Random	Sparse	230
1 batch of 512	Random	Dense	Failed tolerance check

Table 8.3: CUDA CVODE ToR–ORd runtime experiments for 512 cells over the first 20 ms of simulation using relative and absolute tolerances of 10^{-6} and 10^{-8}.

These variations highlight that while adjustments to batch sizes, cell-ordering, and matrix types offer clear pathways for performance tuning, they simultaneously introduce critical trade-offs in numerical stability. Finding the optimal configuration for generalized implicit solvers in parallel environments remains an open optimization challenge that lies outside the scope of this baseline evaluation.

Chapter 9

Conclusions

9.1 Contributions and Key Findings

In this project, we successfully extended `chaste-codegen` and the Chaste simulation framework to support automatically generated CUDA implementations of cardiac cell models. We introduced a functional GPU execution framework for Forward Euler (FE), Backward Euler (BE), and CVODE solvers, bridging the gap between automated high-level code generation and mature, pre-existing cardiac simulation libraries.

To our knowledge, this represents the first highly general toolkit capable of providing a complete end-to-end workflow from arbitrary abstract CellML descriptions to functional GPU device code. Because our pipeline derives device code automatically from abstract metadata, it stands alone in offering this degree of autogeneration and generality, remaining inherently future-proof as increasingly complex biophysical models are developed by the community. Furthermore, we have successfully developed a systematic translation mechanism that takes arbitrary fixed-step size numerical methods on the CPU (like FE and BE) and maps them into corresponding GPU implementations. Highly specialised GPU cardiac simulation frameworks, such as MonoAlg3D, achieve exceptional performance and likely take the title for the most efficient GPU implementation. However, they rely on the hardcoded integration of specific cellular models solved via uniform numerical schemes, such as a hand-tuned explicit Rush–Larsen method. While these specialized products benefit from aggressive optimization, they lack programmatic generality.

The experimental evaluation confirmed the numerical validity of the framework, with the GPU matching the CPU baseline up to negligible roundoff errors. Performance

benchmarks demonstrated that our generated GPU code provides substantial acceleration over the corresponding CPU implementations for the basic fixed-step methods. However, these generated baseline solvers remain uncompetitive with the highly optimized CPU CVODE library. While our CUDA CVODE implementation was slow in comparison, its integration serves as a crucial proof of concept. It explicitly demonstrates that the autogeneration pipeline can adapt to complex, variable-step numerical schemes on top of our fixed-step methods, providing a vital foundational pathway for future adaptive GPU solvers.

9.2 Limitations and Reflections

Implementing automated code generation within mature software ecosystems like Chaste and `chaste-codegen` presented a significant engineering challenge. A considerable portion of development time was spent ensuring new features integrated cleanly without disrupting pre-existing CPU workflows or breaking established validation frameworks. While our final implementation adopts a highly modular design that fits the host architecture, project time constraints prevented us from preparing these modifications for upstream integration into the production branches.

From a software engineering perspective, navigating the build system infrastructure formed a major personal learning curve. Managing substantial CMake configurations spanning multiple files and utilizing complex macro definitions was entirely outside my prior experience. Learning to integrate a completely new compiler toolchain (`nvcc`) into this large-scale, pre-existing build architecture required a deep dive into large-scale project management.

Our performance evaluation was also constrained by available hardware. The consumer-grade laptop GPU used for testing features a heavily restricted FP64 architectural limit, meaning our double-precision benchmarks do not fully reflect the scaling trends achievable on dedicated HPC accelerators. Crucially, because our framework remains a baseline implementation that prioritizes automated model translation over low-level hand-tuning, we have not yet fully optimized the generated kernels. Execution parameters such as memory layouts, kernel launch configurations, and batching strategies remain only lightly explored, leaving clear scope for optimization.

9.3 Future Work

This project establishes a functional baseline for automatically generating GPU cardiac cell solvers inside Chaste, but several important directions remain for future development.

First, the generated kernels should be profiled in detail using tools such as NVIDIA Nsight in order to identify the dominant bottlenecks, quantify register pressure, and evaluate opportunities for reducing synchronisation and host–device transfer overheads. The current implicit solvers would likely also benefit from integration with vendor libraries such as `cuSOLVER` or `cuSPARSE`.

Second, while the current CVODE integration remains CUDA-specific, the generated Forward and Backward Euler kernels already rely primarily on CUDA/HIP-compatible constructs. Extending the build system and validating the generated code on AMD hardware therefore represents a realistic near-term goal.

Third, the fixed-step translation framework developed in this work should be extended to automatically generate GPU implementations of the existing Rush–Larsen and generalised Rush–Larsen solvers already present in Chaste. The generated `CardiacCellCollection` abstraction should also be evaluated as a direct replacement for existing CPU cardiac cell collections within whole-heart simulations.

Finally, the most promising long-term direction is the development of native GPU adaptive solvers with improved timestep management. The CVODE experiments demonstrated that shared adaptive timesteps can significantly constrain performance in heterogeneous batches. Future adaptive GPU solvers should therefore investigate block-structured or warp-level timestep control in order to mitigate these effects and compete with state-of-the-art CPU and GPU monodomain frameworks.

Appendix A

Example Generated GPU Cell Model Code

This appendix provides representative examples of the automatically generated GPU device code produced by the extended `chaste-codegen` pipeline.

The generated ToR–ORd implementation includes the following automatically generated expressions using the stimulus and capacitance:

```
const double var_membrane__Istim_converted =
    stimulus(var_chaste_interface__environment__time); // uA_per_cm2

const double var_membrane__Istim =
    var_membrane__Istim_converted / capacitance; // microA_per_microF
```

motivating the need for us to include the stimulus and the capacitance as arguments. The remainder of this appendix reproduces the complete generated device implementation header for the Luo–Rudy 1991 Forward Euler model. More complex numerical methods and larger contemporary cardiac cell models generate substantially larger source files and are omitted for brevity.

```
1  #ifndef CELLLUORUDY1991FROMCELLMLKERNELS_HPP_
2  #define CELLLUORUDY1991FROMCELLMLKERNELS_HPP_
3
4  //! @file
5  //!
6  //! This source file was generated from CellML by chaste_codegen version 0.10.6
7  //!
8  //! Model: luo_rudy_1991
9  //!
10 //! Processed by chaste_codegen: https://github.com/ModellingWebLab/chaste-codegen
11 //!     (translator: chaste_codegen, model type: Normal)
12 //! on 2026-05-07 10:04:33
13 //!
14 //! <autogenerated>
15
16 #include "ChasteMathDeviceNamespaces.hpp"
17
```

```
18 template<typename ValueType, typename ParameterAccessor>
19 __device__
20 void CellLuoRudy1991FromCellML::EvaluateYDerivativesDevice(
21     ValueType var_chaste_interface__environment__time,
22     const ValueType y[8],
23     ValueType dy[8],
24     const bool setVoltageDerivativeToZero,
25     const ValueType fixedVoltage,
26     const ParameterAccessor parameters,
27     const ValueType capacitance,
28     const DeviceStimulusFunctor<ValueType> stimulus)
29 {
30     // Inputs:
31     // Time units: millisecond
32     ValueType var_chaste_interface__membrane__V = (setVoltageDerivativeToZero ? fixedVoltage : y[0]);
33     // Units: millivolt; Initial value: -83.853
34     ValueType var_chaste_interface__intracellular_calcium_concentration__Cai = y[1];
35     // Units: millimolar; Initial value: 0.0002
36     ValueType var_chaste_interface__fast_sodium_current_m_gate__m = y[2];
37     // Units: dimensionless; Initial value: 0.00187018
38     ValueType var_chaste_interface__fast_sodium_current_h_gate__h = y[3];
39     // Units: dimensionless; Initial value: 0.9804713
40     ValueType var_chaste_interface__fast_sodium_current_j_gate__j = y[4];
41     // Units: dimensionless; Initial value: 0.98767124
42     ValueType var_chaste_interface__slow_inward_current_d_gate__d = y[5];
43     // Units: dimensionless; Initial value: 0.00316354
44     ValueType var_chaste_interface__slow_inward_current_f_gate__f = y[6];
45     // Units: dimensionless; Initial value: 0.99427859
46     ValueType var_chaste_interface__time_dependent_potassium_current_X_gate__X = y[7];
47     // Units: dimensionless; Initial value: 0.16647703
48
49     // Mathematics
50     ValueType d_dt_chaste_interface_var_membrane__V;
51     const ValueType var_fast_sodium_current_h_gate__alpha_h = ((var_chaste_interface__membrane__V <
       -40) ? (static_cast<ValueType>(0.13500000000000001) * ChasteDevice::Exp<ValueType>(static_cast<
       ValueType>(-11.764705882352942) - static_cast<ValueType>(0.14705882352941177) *
       var_chaste_interface__membrane__V)) : (0)); // per_millisecond
52     const ValueType var_fast_sodium_current_h_gate__beta_h = ((var_chaste_interface__membrane__V <
       -40) ? (310000 * ChasteDevice::Exp<ValueType>(static_cast<ValueType>(0.34999999999999998) *
       var_chaste_interface__membrane__V) + static_cast<ValueType>(3.5600000000000001) * ChasteDevice::
       Exp<ValueType>(static_cast<ValueType>(0.079000000000000001) * var_chaste_interface__membrane__V))
        : (static_cast<ValueType>(7.6923076923076916) / (1 + ChasteDevice::Exp<ValueType>(static_cast<
       ValueType>(-0.96036036036036043) - static_cast<ValueType>(0.0900900900900901) *
       var_chaste_interface__membrane__V)))); // per_millisecond
53     const ValueType d_dt_chaste_interface_var_fast_sodium_current_h_gate__h = (1 -
       var_chaste_interface__fast_sodium_current_h_gate__h) * var_fast_sodium_current_h_gate__alpha_h -
       var_fast_sodium_current_h_gate__beta_h * var_chaste_interface__fast_sodium_current_h_gate__h; //
       1 / millisecond
54     const ValueType var_fast_sodium_current_j_gate__alpha_j = ((var_chaste_interface__membrane__V <
       -40) ? ((static_cast<ValueType>(37.780000000000001) + var_chaste_interface__membrane__V) *
       (-127140 * ChasteDevice::Exp<ValueType>(static_cast<ValueType>(0.24440000000000001) *
       var_chaste_interface__membrane__V) - static_cast<ValueType>(3.4740000000000003e-5) * ChasteDevice
       ::Exp<ValueType>(-static_cast<ValueType>(0.043909999999999998) *
       var_chaste_interface__membrane__V)) / (1 + ChasteDevice::Exp<ValueType>(static_cast<ValueType
       >(24.640530000000002) + static_cast<ValueType>(0.311) * var_chaste_interface__membrane__V))) :
       (0)); // per_millisecond
55     const ValueType var_fast_sodium_current_j_gate__beta_j = ((var_chaste_interface__membrane__V <
       -40) ? (static_cast<ValueType>(0.1212) * ChasteDevice::Exp<ValueType>(-static_cast<ValueType
       >(0.01052) * var_chaste_interface__membrane__V) / (1 + ChasteDevice::Exp<ValueType>(static_cast<
       ValueType>(-5.5312920000000005) - static_cast<ValueType>(0.13780000000000001) *
       var_chaste_interface__membrane__V))) : (static_cast<ValueType>(0.29999999999999999) *
       ChasteDevice::Exp<ValueType>(-static_cast<ValueType>(2.5349999999999999e-7) *
       var_chaste_interface__membrane__V) / (1 + ChasteDevice::Exp<ValueType>(static_cast<ValueType
       >(-3.2000000000000002) - static_cast<ValueType>(0.10000000000000001) *
       var_chaste_interface__membrane__V)))); // per_millisecond
56     const ValueType d_dt_chaste_interface_var_fast_sodium_current_j_gate__j = (1 -
       var_chaste_interface__fast_sodium_current_j_gate__j) * var_fast_sodium_current_j_gate__alpha_j -
       var_fast_sodium_current_j_gate__beta_j * var_chaste_interface__fast_sodium_current_j_gate__j; //
```

```
    1 / millisecond
57  const ValueType var_fast_sodium_current_m_gate__alpha_m = (((var_chaste_interface__membrane__V >=
    static_cast<ValueType>(-47.130001)) && (var_chaste_interface__membrane__V <= static_cast<
    ValueType>(-47.129999000000005))) ? (-static_cast<ValueType>(3.2000000000092018e-7) / (1 -
    ChasteDevice::Exp<ValueType>(static_cast<ValueType>(1.0000000000287557e-7))) + static_cast<
    ValueType>(499999.99998562218) * (static_cast<ValueType>(47.130001) +
    var_chaste_interface__membrane__V) * (static_cast<ValueType>(3.2000000000092018e-7) / (1 -
    ChasteDevice::Exp<ValueType>(static_cast<ValueType>(1.0000000000287557e-7))) + static_cast<
    ValueType>(3.2000000000092018e-7) / (1 - ChasteDevice::Exp<ValueType>(static_cast<ValueType
    >(-1.0000000000287557e-7))))) : (static_cast<ValueType>(0.32000000000000001) * (static_cast<
    ValueType>(47.130000000000003) + var_chaste_interface__membrane__V) / (1 - ChasteDevice::Exp<
    ValueType>(static_cast<ValueType>(-4.7130000000000001) - static_cast<ValueType
    >(0.10000000000000001) * var_chaste_interface__membrane__V)))); // per_millisecond
58  const ValueType var_fast_sodium_current_m_gate__beta_m = static_cast<ValueType
    >(0.080000000000000002) * ChasteDevice::Exp<ValueType>(-static_cast<ValueType
    >(0.090909090909090912) * var_chaste_interface__membrane__V); // per_millisecond
59  const ValueType d_dt_chaste_interface_var_fast_sodium_current_m_gate__m = (1 -
    var_chaste_interface__fast_sodium_current_m_gate__m) * var_fast_sodium_current_m_gate__alpha_m -
    var_fast_sodium_current_m_gate__beta_m * var_chaste_interface__fast_sodium_current_m_gate__m; //
    1 / millisecond
60  const ValueType var_slow_inward_current__E_si = static_cast<ValueType>(7.7000000000000002) -
    static_cast<ValueType>(13.028700000000001) * ChasteDevice::Log<ValueType>(
    var_chaste_interface__intracellular_calcium_concentration__Cai); // millivolt
61  const ValueType var_slow_inward_current_d_gate__alpha_d = static_cast<ValueType
    >(0.095000000000000001) * ChasteDevice::Exp<ValueType>(static_cast<ValueType
    >(0.050000000000000003) - static_cast<ValueType>(0.01) * var_chaste_interface__membrane__V) / (1
    + ChasteDevice::Exp<ValueType>(static_cast<ValueType>(0.35999999999999999) - static_cast<
    ValueType>(0.071999999999999995) * var_chaste_interface__membrane__V)); // per_millisecond
62  const ValueType var_slow_inward_current_d_gate__beta_d = static_cast<ValueType
    >(0.070000000000000007) * ChasteDevice::Exp<ValueType>(static_cast<ValueType>(-0.748) -
    static_cast<ValueType>(0.017000000000000001) * var_chaste_interface__membrane__V) / (1 +
    ChasteDevice::Exp<ValueType>(static_cast<ValueType>(2.2000000000000002) + static_cast<ValueType
    >(0.050000000000000003) * var_chaste_interface__membrane__V)); // per_millisecond
63  const ValueType d_dt_chaste_interface_var_slow_inward_current_d_gate__d = (1 -
    var_chaste_interface__slow_inward_current_d_gate__d) * var_slow_inward_current_d_gate__alpha_d -
    var_slow_inward_current_d_gate__beta_d * var_chaste_interface__slow_inward_current_d_gate__d; //
    1 / millisecond
64  const ValueType var_slow_inward_current_f_gate__alpha_f = static_cast<ValueType>(0.012) *
    ChasteDevice::Exp<ValueType>(static_cast<ValueType>(-0.224) - static_cast<ValueType
    >(0.0080000000000000002) * var_chaste_interface__membrane__V) / (1 + ChasteDevice::Exp<ValueType
    >(static_cast<ValueType>(4.2000000000000002) + static_cast<ValueType>(0.14999999999999999) *
    var_chaste_interface__membrane__V)); // per_millisecond
65  const ValueType var_slow_inward_current_f_gate__beta_f = static_cast<ValueType
    >(0.0064999999999999997) * ChasteDevice::Exp<ValueType>(static_cast<ValueType
    >(-0.59999999999999998) - static_cast<ValueType>(0.02) * var_chaste_interface__membrane__V) / (1
    + ChasteDevice::Exp<ValueType>(-6 - static_cast<ValueType>(0.20000000000000001) *
    var_chaste_interface__membrane__V)); // per_millisecond
66  const ValueType d_dt_chaste_interface_var_slow_inward_current_f_gate__f = (1 -
    var_chaste_interface__slow_inward_current_f_gate__f) * var_slow_inward_current_f_gate__alpha_f -
    var_slow_inward_current_f_gate__beta_f * var_chaste_interface__slow_inward_current_f_gate__f; //
    1 / millisecond
67  const ValueType var_slow_inward_current__i_si = (-var_slow_inward_current__E_si +
    var_chaste_interface__membrane__V) * parameters[0] *
    var_chaste_interface__slow_inward_current_d_gate__d *
    var_chaste_interface__slow_inward_current_f_gate__f; // microA_per_cm2
68  const ValueType d_dt_chaste_interface_var_intracellular_calcium_concentration__Cai = static_cast<
    ValueType>(7.0000000000000007e-6) - static_cast<ValueType>(0.070000000000000007) *
    var_chaste_interface__intracellular_calcium_concentration__Cai - static_cast<ValueType>(0.0001) *
     var_slow_inward_current__i_si; // millimolar / millisecond
69  const ValueType var_time_dependent_potassium_current_X_gate__alpha_X = static_cast<ValueType
    >(0.0005000000000000001) * ChasteDevice::Exp<ValueType>(static_cast<ValueType
    >(4.1500000000000004) + static_cast<ValueType>(0.083000000000000004) *
    var_chaste_interface__membrane__V) / (1 + ChasteDevice::Exp<ValueType>(static_cast<ValueType
    >(2.8500000000000001) + static_cast<ValueType>(0.057000000000000002) *
    var_chaste_interface__membrane__V)); // per_millisecond
70  const ValueType var_time_dependent_potassium_current_X_gate__beta_X = static_cast<ValueType
    >(0.0012999999999999999) * ChasteDevice::Exp<ValueType>(static_cast<ValueType>(-1.2) -
    static_cast<ValueType>(0.059999999999999998) * var_chaste_interface__membrane__V) / (1 +
```

```
    ChasteDevice::Exp<ValueType>(static_cast<ValueType>(-0.80000000000000004) - static_cast<ValueType
    >(0.040000000000000001) * var_chaste_interface__membrane__V)); // per_millisecond
71  const ValueType d_dt_chaste_interface_var_time_dependent_potassium_current_X_gate__X = (1 -
    var_chaste_interface__time_dependent_potassium_current_X_gate__X) *
    var_time_dependent_potassium_current_X_gate__alpha_X -
    var_chaste_interface__time_dependent_potassium_current_X_gate__X *
    var_time_dependent_potassium_current_X_gate__beta_X; // 1 / millisecond
72
73  if (setVoltageDerivativeToZero)
74  {
75      d_dt_chaste_interface_var_membrane__V = 0.0;
76
77  }
78  else
79  {
80      const ValueType var_background_current__E_b = static_cast<ValueType>(-59.869999999999997);
81      const ValueType var_background_current__g_b = static_cast<ValueType>(0.039210000000000002);
82      const ValueType var_ionic_concentrations__Ki = 145;
83      const ValueType var_ionic_concentrations__Ko = static_cast<ValueType>(5.4000000000000004);
84      const ValueType var_ionic_concentrations__Nai = 18;
85      const ValueType var_ionic_concentrations__Nao = 140;
86      const ValueType var_membrane__C = 1;
87      const ValueType var_membrane__F = static_cast<ValueType>(96484.600000000006);
88      const ValueType var_membrane__I_stim = stimulus(var_chaste_interface__environment__time);
89      const ValueType var_membrane__R = 8314;
90      const ValueType var_membrane__T = 310;
91      const ValueType var_fast_sodium_current__E_Na = var_membrane__R * var_membrane__T *
    ChasteDevice::Log<ValueType>(var_ionic_concentrations__Nao / var_ionic_concentrations__Nai) /
    var_membrane__F;
92      const ValueType var_background_current__i_b = (-var_background_current__E_b +
    var_chaste_interface__membrane__V) * var_background_current__g_b;
93      const ValueType var_fast_sodium_current__i_Na = ChasteDevice::Pow<ValueType>(
    var_chaste_interface__fast_sodium_current_m_gate__m, 3) * (-var_fast_sodium_current__E_Na +
    var_chaste_interface__membrane__V) * parameters[1] *
    var_chaste_interface__fast_sodium_current_h_gate__h *
    var_chaste_interface__fast_sodium_current_j_gate__j;
94      const ValueType var_plateau_potassium_current__Kp = 1 / (1 + ChasteDevice::Exp<ValueType>(
    static_cast<ValueType>(1.2521739130434781) - static_cast<ValueType>(0.16722408026755853) *
    var_chaste_interface__membrane__V));
95      const ValueType var_plateau_potassium_current__g_Kp = static_cast<ValueType>(0.0183);
96      const ValueType var_time_dependent_potassium_current__PR_NaK = static_cast<ValueType
    >(0.018329999999999999);
97      const ValueType var_time_dependent_potassium_current__E_K = var_membrane__R * var_membrane__T
    * ChasteDevice::Log<ValueType>((var_ionic_concentrations__Nao *
    var_time_dependent_potassium_current__PR_NaK + var_ionic_concentrations__Ko) / (
    var_ionic_concentrations__Nai * var_time_dependent_potassium_current__PR_NaK +
    var_ionic_concentrations__Ki)) / var_membrane__F;
98      const ValueType var_time_dependent_potassium_current__g_K = static_cast<ValueType
    >(0.43033148291193518) * ChasteDevice::Sqrt<ValueType>(var_ionic_concentrations__Ko) * parameters
    [2];
99      const ValueType var_time_dependent_potassium_current_Xi_gate__Xi = ((
    var_chaste_interface__membrane__V > -100) ? (static_cast<ValueType>(2.8370000000000002) * (-1 +
    ChasteDevice::Exp<ValueType>(static_cast<ValueType>(3.0800000000000001) + static_cast<ValueType
    >(0.040000000000000001) * var_chaste_interface__membrane__V)) / ((77 +
    var_chaste_interface__membrane__V) * ChasteDevice::Exp<ValueType>(static_cast<ValueType
    >(1.4000000000000001) + static_cast<ValueType>(0.040000000000000001) *
    var_chaste_interface__membrane__V))) : (1));
100     const ValueType var_time_dependent_potassium_current__i_K = (-
    var_time_dependent_potassium_current__E_K + var_chaste_interface__membrane__V) *
    var_time_dependent_potassium_current__g_K *
    var_chaste_interface__time_dependent_potassium_current_X_gate__X *
    var_time_dependent_potassium_current_Xi_gate__Xi;
101     const ValueType var_time_independent_potassium_current__E_K1 = var_membrane__R *
    var_membrane__T * ChasteDevice::Log<ValueType>(var_ionic_concentrations__Ko /
    var_ionic_concentrations__Ki) / var_membrane__F;
102     const ValueType var_plateau_potassium_current__E_Kp =
    var_time_independent_potassium_current__E_K1;
103     const ValueType var_plateau_potassium_current__i_Kp = (-var_plateau_potassium_current__E_Kp +
```

```
    var_chaste_interface__membrane__V) * var_plateau_potassium_current__Kp *
    var_plateau_potassium_current__g_Kp;
104     const ValueType var_time_independent_potassium_current__g_K1 = static_cast<ValueType
    >(0.26022144771684724) * ChasteDevice::Sqrt<ValueType>(var_ionic_concentrations__Ko);
105     const ValueType var_time_independent_potassium_current_K1_gate__alpha_K1 = static_cast<
    ValueType>(1.02) / (1 + ChasteDevice::Exp<ValueType>(static_cast<ValueType>(-14.1227775) +
    static_cast<ValueType>(0.23849999999999999) * var_chaste_interface__membrane__V - static_cast<
    ValueType>(0.23849999999999999) * var_time_independent_potassium_current__E_K1));
106     const ValueType var_time_independent_potassium_current_K1_gate__beta_K1 = (ChasteDevice::Exp<
    ValueType>(static_cast<ValueType>(-36.698642499999998) + static_cast<ValueType
    >(0.061749999999999999) * var_chaste_interface__membrane__V - static_cast<ValueType
    >(0.061749999999999999) * var_time_independent_potassium_current__E_K1) + static_cast<ValueType
    >(0.49124000000000001) * ChasteDevice::Exp<ValueType>(static_cast<ValueType>(0.43983232) +
    static_cast<ValueType>(0.080320000000000003) * var_chaste_interface__membrane__V - static_cast<
    ValueType>(0.080320000000000003) * var_time_independent_potassium_current__E_K1)) / (1 +
    ChasteDevice::Exp<ValueType>(static_cast<ValueType>(-2.4444678999999998) + static_cast<ValueType
    >(0.51429999999999998) * var_time_independent_potassium_current__E_K1 - static_cast<ValueType
    >(0.51429999999999998) * var_chaste_interface__membrane__V));
107     const ValueType var_time_independent_potassium_current_K1_gate__K1_infinity =
    var_time_independent_potassium_current_K1_gate__alpha_K1 / (
    var_time_independent_potassium_current_K1_gate__alpha_K1 +
    var_time_independent_potassium_current_K1_gate__beta_K1);
108     const ValueType var_time_independent_potassium_current__i_K1 = (-
    var_time_independent_potassium_current__E_K1 + var_chaste_interface__membrane__V) *
    var_time_independent_potassium_current__g_K1 *
    var_time_independent_potassium_current_K1_gate__K1_infinity;
109     d_dt_chaste_interface_var_membrane__V = -(var_background_current__i_b +
    var_fast_sodium_current__i_Na + var_membrane__I_stim + var_plateau_potassium_current__i_Kp +
    var_slow_inward_current__i_si + var_time_dependent_potassium_current__i_K +
    var_time_independent_potassium_current__i_K1) / var_membrane__C;
110
111   }
112   dy[0] = d_dt_chaste_interface_var_membrane__V;
113   dy[1] = d_dt_chaste_interface_var_intracellular_calcium_concentration__Cai;
114   dy[2] = d_dt_chaste_interface_var_fast_sodium_current_m_gate__m;
115   dy[3] = d_dt_chaste_interface_var_fast_sodium_current_h_gate__h;
116   dy[4] = d_dt_chaste_interface_var_fast_sodium_current_j_gate__j;
117   dy[5] = d_dt_chaste_interface_var_slow_inward_current_d_gate__d;
118   dy[6] = d_dt_chaste_interface_var_slow_inward_current_f_gate__f;
119   dy[7] = d_dt_chaste_interface_var_time_dependent_potassium_current_X_gate__X;
120 }
121 #endif // CELLLUORUDY1991FROMCELLMLKERNELS_HPP_
```

Appendix B

Implementation of the GPU Newton Solver

This appendix provides the `CardiacNewtonSolverDevice` used by the Backward Euler solver to solve the reduced nonlinear system in Equation 2.19.

```
template<typename CellType, typename ValueType>
struct CardiacNewtonSolverCuda
{
public:
    static constexpr unsigned NONLINEAR_SIZE = CellType::NONLINEAR_SIZE;

    template<typename ParameterAccessor>
    __device__
    static void Solve(ValueType guess[NONLINEAR_SIZE],
                      const ValueType y_old[/*TOTAL_SIZE*/],
                      const ValueType dt,
                      const ValueType time,
                      const ParameterAccessor parameters,
                      const bool setVoltageDerivativeToZero,
                      const ValueType fixedVoltage,
                      ValueType residual[NONLINEAR_SIZE],
                      ValueType Jmat[NONLINEAR_SIZE][NONLINEAR_SIZE],
                      ValueType update[NONLINEAR_SIZE],
                      const ValueType capacitance,
                      const DeviceStimulusFunctor<ValueType> stimulus)
    {
        unsigned counter = 0;
        const ValueType eps = static_cast<ValueType>(1e-6);

        // Evaluate R_z(u^*_{n+1}, z^{(k)})
        CellType::ComputeResidualDevice(...);

        ValueType norm_of_residual = norm_inf(residual);
        if (isnan(norm_of_residual)) return;
        ValueType norm_of_update = static_cast<ValueType>(0);
```

```
        do {
            // Form J_{zz}^{(k)}
            CellType::ComputeJacobianDevice(...);
            // Solve J_{zz}^{(k)} delta z^{(k)} = R_z(u^*_{n+1}, z^{(k)})
            SolveLinearSystem(Jmat, residual, update);

            norm_of_update = norm_inf(update);

            // Apply z^{(k+1)} = z^{(k)} - delta z^{(k)}
            for (unsigned i = 0; i < NONLINEAR_SIZE; i++) {
                guess[i] -= update[i];
            }
            ValueType prev_res = norm_of_residual;

            // Re-evaluate R_z(u^*_{n+1}, z^{(k+1)})
            CellType::ComputeResidualDevice(...);
            norm_of_residual = norm_inf(residual);

            // If the residual increased, apply a simple damping correction
            if (norm_of_residual > prev_res && norm_of_update > eps) {
                ValueType relative_change_max = 0;
                unsigned direction = 0;

                for (unsigned i = 0; i < NONLINEAR_SIZE; i++) {
                    ValueType rc = fabs(update[i] / guess[i]);

                    if (rc > relative_change_max) {
                        relative_change_max = rc;
                        direction = i;
                    }
                }

                if (relative_change_max > static_cast<ValueType>(1.0)) {
                    guess[direction] += static_cast<ValueType>(0.8)
                        * update[direction];

                    CellType::ComputeResidualDevice(...);
                    norm_of_residual = norm_inf(residual);
                }
            }

            counter++;
            // Stop after 15 Newton iterations, matching the CPU implementation
            if (counter > 15) return;

        } while (norm_of_update > eps);
    }

protected:
    __device__ __forceinline__
    static void SolveLinearSystem(
        ValueType J[NONLINEAR_SIZE][NONLINEAR_SIZE],
        ValueType residual[NONLINEAR_SIZE],
        ValueType update[NONLINEAR_SIZE]) {
```

```
        for (unsigned i = 0; i < NONLINEAR_SIZE; ++i) {
            const ValueType diag = J[i][i];

            for (unsigned ii = i + 1; ii < NONLINEAR_SIZE; ++ii) {
                const ValueType fact = J[ii][i] / diag;

                for (unsigned j = i; j < NONLINEAR_SIZE; ++j) {
                    J[ii][j] -= fact * J[i][j];
                }
                residual[ii] -= fact * residual[i];
            }
        }
        for (int i = NONLINEAR_SIZE - 1; i >= 0; --i) {
            ValueType sum = residual[i];

            for (unsigned j = i + 1; j < NONLINEAR_SIZE; ++j) {
                sum -= J[i][j] * update[j];
            }
            update[i] = sum / J[i][i];
        }
    }

    __device__ __forceinline__
    static ValueType norm_inf(const ValueType* x) {
        ValueType maxv = 0;

        for (unsigned i = 0; i < NONLINEAR_SIZE; ++i) {
            maxv = fmax(maxv, fabs(x[i]));
        }
        return maxv;
    }
};
```

Appendix C

Implementation of the GPU-Enabled CVODE Solver

This appendix contains selected implementation details of the GPU-enabled CVODE integration introduced in Section 7.4. The implementation consists primarily of two components: the `BatchedCvodeSystem`, which exposes GPU-based right-hand side and Jacobian callbacks through the SUNDIALS interface, and the `CvodeCardiacCellCollection`, which manages batching, CUDA streams, and concurrent execution of independent CVODE solves. The first listing shows the GPU-specific CVODE setup and callback configuration required by the CUDA-enabled SUNDIALS interface.

```
template<typename CellType, typename ValueType>
void BatchedCvodeSystem<CellType, ValueType>::SetupCvode(
    N_Vector initialConditions,
    realtype tStart,
    realtype maxDt)
{
    // Attach initial state vector to CVODE system.
    DeleteVector(mStateVariables);
    mStateVariables = initialConditions;

    // Create or reinitialise the CVODE BDF solver.
    if (!mpCvodeMem) {

        mpCvodeMem = CVodeCreate(
            CV_BDF,
            CvodeContextManager::Instance()->GetSundialsContext());

        CVodeSetUserData(mpCvodeMem, static_cast<void*>(this));

        CVodeInit(
            mpCvodeMem,
            AbstractCvodeSystemRhsAdaptor,
```

```
            tStart,
            initialConditions);

        CVodeSStolerances(mpCvodeMem, mRelTol, mAbsTol);

    } else {

        CVodeReInit(mpCvodeMem, tStart, initialConditions);
    }

    // Configure timestep limits.
    CVodeSetInitStep(mpCvodeMem, 1.e-16);

    if (maxDt > 0) {
        CVodeSetMaxStep(mpCvodeMem, maxDt);
    }

    // Create CUDA-aware cuSPARSE and cuSOLVER handles.
    cusparseCreate(&mCuspHandle);
    cusparseSetStream(mCuspHandle, mStream);

    cusolverSpCreate(&mCusolHandle);
    cusolverSpSetStream(mCusolHandle, mStream);

    // Construct block CSR Jacobian storage.
    const sunindextype nblocks = static_cast<sunindextype>(mBatchSize);

    const sunindextype block_size = static_cast<sunindextype>(mNvarsPerCell);

    const sunindextype nnz_per_block =
        mUseSparse ? CellType::NNZ : block_size * block_size;

    mSUNMatrix = SUNMatrix_cuSparse_NewBlockCSR(
        nblocks,
        block_size,
        block_size,
        nnz_per_block,
        mCuspHandle,
        CvodeContextManager::Instance()->GetSundialsContext());

    // Upload sparse or dense block sparsity pattern.
    sunindextype* d_rowptr =
        SUNMatrix_cuSparse_IndexPointers(mSUNMatrix);

    sunindextype* d_colind =
        SUNMatrix_cuSparse_IndexValues(mSUNMatrix);

    if (mUseSparse) {

        cudaMemcpyAsync(...);

    } else {

        InitialiseDenseBlockPattern(...);
```

```
    }

    cudaStreamSynchronize(mStream);

    SUNMatrix_cuSparse_SetFixedPattern(mSUNMatrix, 1);

    // Attach GPU linear solver and Jacobian callback.
    mBlockJacobianData =
        static_cast<ValueType*>(SUNMatrix_cuSparse_Data(mSUNMatrix));

    mSUNLinSol = SUNLinSol_cuSolverSp_batchQR(
        initialConditions,
        mSUNMatrix,
        mCusolHandle,
        CvodeContextManager::Instance()->GetSundialsContext());

    CVodeSetLinearSolver(mpCvodeMem, mSUNLinSol, mSUNMatrix);

    CVodeSetJacFn(mpCvodeMem, AbstractCvodeSystemJacAdaptor);
}
```

The simplified solve loop below highlights the batched adaptive stepping and host-device synchronisation performed during execution.

```
while (!stepper.IsTimeAtEnd())
{
    // Advance CVODE to the next requested output time.
    CVodeSetStopTime(mpCvodeMem, stepper.GetNextTime());

    double cvode_stopped_at = stepper.GetTime();

    CVode(
        mpCvodeMem,
        stepper.GetNextTime(),
        mStateVariables,
        &cvode_stopped_at,
        CV_NORMAL);

    // Synchronise and copy accepted solution back to host memory.
    cudaStreamSynchronize(mStream);
    N_VCopyFromDevice_Cuda(mStateVariables);

    double* h_data =
        N_VGetHostArrayPointer_Cuda(mStateVariables);

    // Append sampled solution for every cell in the batch.
    for (unsigned i = 0; i < mBatchSize; ++i) {

        ...
    }

    stepper.AdvanceOneTimeStep();
}
```

Declaration of Generative AI Use

In accordance with the University and Departmental guidelines on good academic practice, I declare the iterative use of generative artificial intelligence software tools across multiple phases of this project's development:

1. **Build System Infrastructure:** In early November 2025, I used a personal ChatGPT Plus account running the GPT-5.1 model to generate prototype CMake configurations. This initial AI-generated output was mostly discarded and rewritten by hand (although much boilerplate remains) as I developed a better understanding of the CMake build system. The final build infrastructure represents my own work.

2. **Forward Euler Core Implementation:** Despite the order of presentation, I actually implemeneted the Backward Euler method first along with the `CardiacCellCollection`. I iteratively used a personal OpenAI Codex account utilizing the GPT-5.4-High model to develop the Forward Euler GPU solver implementation using this infrastructure and mirroring the pipeline used for the hand written Backward Euler method.

3. **Automated Test Generation:** I utilized GPT-5.4-High within Codex to expand upon my pre-existing, hand-written correctness tests and benchmark scripts originally developed for the Backward Euler implementation. Using these established files as a foundation, I leveraged the model to build out the test suites responsible for verifying the remaining models under varied parameter spaces and outputting the raw simulation data. To minimize compilation overhead, these tests were structured across multiple distinct files. To streamline development, I provided the model with the functional specifications mapping the new API to the required file outputs, generating the multi-file boilerplate framework within a single prompt rather than constructing the individual configurations via separate manual commands. I then iteratively prompted the model to adjust step lengths and error bounds for the CVODE tests while independently verifying the mathematical correctness of the resulting test routines.

4. **Data Visualization and Plotting:** I utilized Codex (GPT-5.4-High) to generate the data visualization scripts. I designed the initial plot configurations

(including axis definitions and data mappings), while the AI tool was responsible for executing granular cosmetic modifications, including spacing, formatting, and color palettes, without direct manual code input from me.

5. **Prose Editing and Reductions:** I used my ChatGPT Plus account running the GPT-5.5 model during to help cut down my final draft to beneath the word limit, I worked with it iteratively, presenting it with what text I had already written and then implementing some of the changes that it suggested.

All other components of this report represent my original work.

Bibliography

[1] AMD. HIP C++ Language Support, 2024. Accessed: 2026-03-15.

[2] Lucas Arantes Berg, Rafael Oliveira, Julià Camps, Zhinuo Wang, Ruben Doste, Alfonso Bueno-Orovio, Rodrigo Santos, and Blanca Rodriguez. MonoAlg3D: Enabling Cardiac Electrophysiology Digital Twins with an Efficient Open Source Scalable Solver on GPU Clusters. 04 2025.

[3] Cody J Balos, David J Gardner, Carol S Woodward, and Daniel R Reynolds. Enabling GPU accelerated computing in the SUNDIALS time integration library. *Parallel Computing*, 108:102836, 2021.

[4] Nathan Bell and Jared Hoberock. Chapter 26 - Thrust: A Productivity-Oriented Library for CUDA. In Wen mei W. Hwu, editor, *GPU Computing Gems Jade Edition*, Applications of GPU Computing Series, pages 359–371. Morgan Kaufmann, Boston, 2012.

[5] Michael Clerx, Pieter Collins, Enno de Lange, and Paul G. A. Volders. Myokit: A simple interface to cardiac cellular electrophysiology. *Progress in Biophysics and Molecular Biology*, 120(1-3):100–114, 2016.

[6] Fergus R. Cooper, Ruth E. Baker, Miguel O. Bernabeu, Rafel Bordas, Louise Bowler, Alfonso Bueno-Orovio, Helen M. Byrne, Valentina Carapella, Louie Cardone-Noott, Cooper Jonatha, Sara Dutta, Benjamin D. Evans, Alexander G. Fletcher, James A. Grogan, Wenxian Guo, Daniel G. Harvey, Maurice Hendrix, David Kay, Jochen Kursawe, Philip K. Maini, Beth McMillan, Gary R. Mirams, James M. Osborne, Pras Pathmanathan, Joe M. Pitt-Francis, Martin Robinson, Blanca Rodriguez, Raymond J. Spiteri, and David J. Gavaghan. Chaste: Cancer, Heart and Soft Tissue Environment. *Journal of Open Source Software*, 5(47):1848, 2020.

[7] CxxTest Development Team. CxxTest Unit Testing Framework. `https://github.com/cxxtest/cxxtest`. Accessed: 2026-05-07.

[8] Gregory M. Faber and Yoram Rudy. Action Potential and Contractility in [Na+]i Overloaded Cardiac Myocytes. *Biophysical Journal*, 78(5):2392–2404, 2000.

[9] David J Gardner, Daniel R Reynolds, Carol S Woodward, and Cody J Balos. Enabling new flexibility in the SUNDIALS suite of nonlinear and differential/algebraic equation solvers. *ACM Transactions on Mathematical Software (TOMS)*, 2022.

[10] Alan Garny and Peter J. Hunter. OpenCOR: a modular and interoperable approach to computational biology. *Frontiers in Physiology*, 6:26, 2015.

[11] M. Hendrix, M. Clerx, A. U. Tamuri, et al. cellmlmanip and chaste_codegen: automatic CellML to C++ code generation with fixes for singularities and automatically generated Jacobians [version 2; peer review: 3 approved]. *Wellcome Open Research*, 6:261, 2022.

[12] A. L. Hodgkin and A. F. Huxley. A quantitative description of membrane current and its application to conduction and excitation in nerve. *The Journal of Physiology*, 117(4):500–544, 08 10.1113/jphysiol.1952.sp004764.

[13] Kevin Hunter, Lawrence Spracklen, and Subutai Ahmad. Two Sparsities Are Better Than One: Unlocking the Performance Benefits of Sparse-Sparse Networks, 12 2021.

[14] Kitware, Inc. CMake. `https://cmake.org/`. Accessed: 2026-05-07.

[15] Catherine Lloyd, James Lawson, Peter Hunter, and Poul Nielsen. The CellML model repository. *Bioinformatics (Oxford, England)*, 24:2122–3, 09 2008.

[16] Ching-Hsing Luo and Yoram Rudy. A model of the ventricular cardiac action potential: Depolarization, repolarization, and their interaction. *Circulation Research*, 68(6):1501–1526, June 1991.

[17] NVIDIA. NVIDIA Blackwell Datasheet. `https://nvdam.widen.net/s/wwnsxrhm2w/blackwell-datasheet-3384703`, 2025. Accessed: 2026-05-10.

[18] NVIDIA Corporation. *CUDA C++ Programming Guide*. NVIDIA Corporation, 2025. Version 13.1. Accessed: 2025-12-10.

[19] NVIDIA Corporation. NVIDIA RTX Blackwell GPU Architecture: Built for Neural Rendering. Technical report, NVIDIA Corporation, 2025. Accessed: 2025-12-01.

[20] Pallets Projects. Jinja2, 2024. Accessed: 2026-05-12.

[21] John Pennycook, S.D. Hammond, Steven Wright, Andy Herdman, I. Miller, and Stephen Jarvis. An investigation of the performance portability of OpenCL. *Journal of Parallel and Distributed Computing*, 73:1439–1450, 11 2013.

[22] Gernot Plank, Axel Loewe, Aurel Neic, Christoph Augustin, Yung-Lin (Cary) Huang, Matthias Gsell, Elias Karabelas, Mark Nothstein, Jorge Sánchez, Anton Prassl, Gunnar Seemann, and Ed Vigmond. The openCARP Simulation Environment for Cardiac Electrophysiology. *Computer Methods and Programs in Biomedicine*, 208:106223, 2021.

[23] Stanley Rush and Hugh Larsen. A Practical Algorithm for Solving Dynamic Membrane Equations. *IEEE Transactions on Biomedical Engineering*, BME-25(4):389–392, 1978.

[24] Jakub Tomek, Alfonso Bueno-Orovio, Elisa Passini, Xin Zhou, Ana Minchole, Oliver Britton, Chiara Bartolucci, Stefano Severi, Alvin Shrier, Laszlo Virag, Andras Varro, and Blanca Rodriguez. Development, calibration, and validation of a novel human ventricular myocyte model in health, disease, and drug block. *eLife*, 8:e48890, dec 2019.

[25] Jose Vicente, Robbert Zusterzeel, Lars Johannesen, Jay Mason, Philip Sager, Vaishali Patel, Murali K. Matta, Zhihua Li, Jianing Liu, Christine Garnett, Norman Stockbridge, Issam Zineh, and David G. Strauss. Mechanistic Model-Informed Proarrhythmic Risk Assessment of Drugs: Review of the "CiPA" Initiative and Design of a Prospective Clinical Validation Study. *Clinical Pharmacology & Therapeutics*, 103(1):54–66, 2018.

[26] J.P. Whiteley. An Efficient Numerical Technique for the Solution of the Monodomain and Bidomain Equations. *IEEE Transactions on Biomedical Engineering*, 53(11):2139–2147, 2006.